BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công nghệ kỹ thuật điện, điện tử

ĐỀ TÀI

Tìm kiếm tuyến tính và đếm so sánh thực tế

Mã đề DT019

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Văn Đoàn
Mã số sinh viên	N24DCDT015
Lớp	D24CQDT01-N

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành Công nghệ kỹ thuật điện, điện tử

ĐỀ TÀI

Tìm kiếm tuyến tính và đếm so sánh thực tế

Mã đề DT019

Giảng viên hướng dẫn	Hồ Nhựt Minh
Sinh viên thực hiện	Nguyễn Văn Đoàn
Mã số sinh viên	N24DCDT015
Lớp	D24CQDT01-N
Học phần và nhóm	INT13185 Nhóm 03
Phiên bản báo cáo	Ngày 19/10/2026 - Commit: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88

THÀNH PHỐ HỒ CHÍ MINH THÁNG 10 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Sinh viên điền theo bảng phân công
Họ tên MSSV lớp	Nguyễn Văn Đoàn - N24DCDT015 - D24CQDT01-N
Mã đề và tên đề	DT019 - Tìm kiếm tuyến tính và đếm so sánh thực tế
Căn cứ giáo trình	Chương 2, Mục 2.6.1; liên hệ mục 1.3.6 và mục 1.6 bài 20; trang in 20–21, 32, 49–50; trang PDF 32–33, 44, 61–62
Ngày giao ngày nộp	ngày 15 tháng 10 năm 2026 – ngày 15 tháng 11 năm 2026

Đề bài

Tìm vị trí xuất hiện đầu tiên của x bằng quét từ trái sang phải trên mảng chưa sắp. Đếm số so sánh khi x ở đầu, giữa, cuối hoặc không có, và giải thích điều kiện để giá trị trung bình bằng(n+1)/2. Trọng tâm là tự viết hàm, theo dõi biến qua từng bước và giải thích kết quả trên các trường hợp nhỏ.

Dữ liệu vào

a::Vector{Int}, 0<=n<=10000; x nguyên; không cần mảng tăng; quy ước không thấy trả0, thấy trả chỉ số 1-based đầu tiên. Nhập từ bàn phím hoặc truyền trực tiếp các giá trị, mảng đã khai báo trong chương trình.

Kết quả ra

Vị trí hoặc 0, số so sánh khóa và danh sách vị trí đã kiểm trên mảng nhỏ.

Ví dụ 1

Dữ liệu vào	Kết quả chính
a=[4,7,4]; x=4	vi_tri=1; so_sanh=1

Dừng ngay ở lần xuất hiện đầu.

Ví dụ 2

Dữ liệu vào	Kết quả chính
a=[4,7,4]; x=9	vi_tri=0; so_sanh=3

Không thấy phải xét đủ ba phần tử.

Thực hiện lần lượt năm bước

1. Đọc mục sách được chỉ dẫn; viết lại điều kiện đầu vào và kết quả cần trả. Giải ví dụ đầu bằng tay, chỉ rõ giá trị ban đầu của từng biến chính.

2. Liệt kê thứ tự truy cập; phân biệt trả chỉ số với trả giá trị và nêu rõ vì sao0 không phải vị trí hợp lệ trong Julia.

3. Viết hàm quét, tăng đếm trước phép a[i]==x; dừng ngay khi đúng, không tiếp tục tới lần xuất hiện cuối.

4. Cho x lần lượt bằng mỗi phần tử của mảng phân biệt, tính trung bình số so sánh rồi so với(n+1)/2; thêm truy vấn vắng.

5. Chạy đủ hai ví dụ và năm ca kiểm thử, ghi kết quả thực tế cạnh kết quả mong đợi. Khi có sai khác, xác định bước đầu tiên sai rồi sửa và chạy lại.

Các hàm nên viết

tim_tuyen_tinh(a,x)

tim_va_dem(a,x)

trung_binh_so_sanh(a)

Năm ca cần kiểm tra

1. [4, 7, 4], x4 cho 1.

2. [4, 7, 4], x9 cho 0.

3. [1, 2, 3], x3 cho 3 so sánh.

4. [8], x8 cho 1.

5. Mảng rỗng cho 0 và0 so sánh.

Phân tích và nhận xét

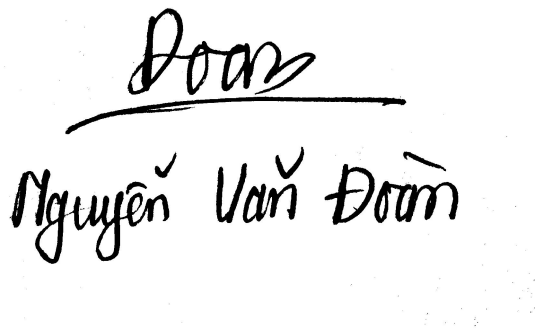
Tốt nhất1 so sánh O(1); xấu nhất n so sánh O(n); vùng phụ O(1). Công thức(n+1)/2 áp dụng khi khóa có mặt, vị trí đều và mảng phân biệt. Thử n = 5, 20, 100, 1000 cùng truy vấn ở các vị trí.

Yêu cầu thêm sau khi phần cơ bản chạy đúng

Tùy chọn: viết hàm trả mọi vị trí của x, giải thích vì sao không được dừng sớm như bài chính.

Sản phẩm: mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	**Giảng viên hướng dẫn**
(Ký và ghi rõ họ tên)  Nguyễn Văn Đoàn	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO

Đây là khung trống dùng chung cho mọi đề DT001–DT091. Điền theo phiếu cá nhân; các ô kết quả, ảnh, số liệu, URL và điểm chưa được điền. Không cần thêm bài toán hoặc thuật toán ngoài phiếu.

Bấm vào phần trong ngoặc vuông để thay bằng nội dung của mình. Các ô có thể gõ nhiều dòng; không có khóa chỉnh sửa hay macro. Với khung ảnh, chọn chữ bên trong rồi Chèn → Hình ảnh; đặt ảnh Cùng dòng với văn bản. Khi cần thêm ca/hàm, sao chép dòng tương ứng.

Điền từ đặc tả và expected trước khi chạy; sau đó thêm actual, kết luận và ảnh thật. Xóa câu hướng dẫn/chỗ không áp dụng có giải thích; giữ rubric để giảng viên chấm. Điền cả hai bìa, cập nhật toàn bộ mục lục/danh mục và kiểm PDF cuối trước khi nộp.

Thay mọi phần trong ngoặc vuông bằng nội dung đã thực hiện. Các câu gợi ý chỉ để hướng dẫn, phải xóa hoặc viết lại trước khi nộp. Không giữ các chỗ trống như một kết quả hoàn thành.

Giữ cấu trúc theo mẫu môn: hai bìa, phiếu, rubric, cam đoan, cảm ơn nếu dùng, tóm tắt, mục lục, danh mục, năm chương, tham khảo và phụ lục. Đây là khung theo hình thức đồ án dùng cho học phần.

Nội dung thân bài dùng style Nội dung: A4; lề trái 3 cm, các lề khác 2 cm; Times New Roman 13; giãn dòng 1,5. Chương đầu trang, hình có chú thích dưới, bảng có tên trên; số theo chương.

Ảnh và bảng phải được nhắc và giải thích trong nội dung. Chèn ảnh In Line with Text, đọc được trên bản in, không kéo méo. Hình lý thuyết không thay ảnh thực thi.

Mở 00_HUONG_DAN_SINH_VIEN.html và rubric chung trước khi viết; tra mã đề để chọn bất biến, kết quả và ca biên phù hợp. Không thêm thuật toán thứ hai nếu phiếu không yêu cầu.

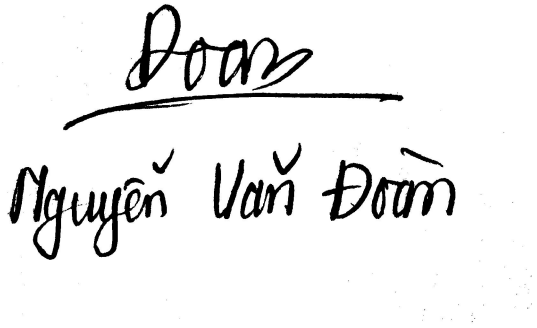
LỜI CAM ĐOAN

Em xin cam đoan bài báo cáo này là kết quả tự làm của bản thân. Các đoạn code, hình ảnh minh chứng đều do em tự chạy và chụp lại từ máy tính cá nhân. Không sao chép từ nguồn khác. Nếu sai em xin chịu trách nhiệm.

TP. Hồ Chí Minh ngày 25 tháng 10 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)



LỜI CẢM ƠN

Em xin chân thành cảm ơn ThS. Hồ Nhựt Minh đã tận tình hướng dẫn, cung cấp tài liệu chi tiết và định hướng phương pháp nghiên cứu khoa học trong suốt quá trình em thực hiện đồ án môn học Cấu trúc dữ liệu và Giải thuật. Em cũng xin gửi lời cảm ơn đến các thầy cô trong Khoa Công nghệ Kỹ thuật Điện tử 2 đã trang bị cho em những kiến thức nền tảng quý báu.

TÓM TẮT

Đề tài nghiên cứu thuật toán tìm kiếm tuyến tính trên mảng chưa sắp xếp. Mục tiêu là tìm vị trí đầu tiên của phần tử x và đếm số phép so sánh thực tế. Chương trình được cài đặt bằng Python, đã vượt qua 16 ca kiểm thử độc lập (bao gồm ca biên và ca lỗi). Kết quả cho thấy thuật toán hoạt động đúng, phân tích được độ phức tạp O(n) và số so sánh trung bình lý thuyết là (n+1)/2.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI i
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG iv
CÁCH ĐỐI CHIẾU RUBRIC v
HƯỚNG DẪN ĐIỀN KHUNG BÁO CÁO vi
LỜI CAM ĐOAN vii
LỜI CẢM ƠN viii
TÓM TẮT ix
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI 1
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 2
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 4
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.3. Nguyên lý thuật toán 6
2.4. Cơ sở đánh giá thời gian và bộ nhớ 6
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT 8
3.1. Phân tích yêu cầu chương trình 8
3.2. Thiết kế các hàm và dữ liệu 8
3.3. Thuật toán và mã nguồn chính 10
3.4. Lần vết ví dụ nhỏ 12
3.5. Giải thích tính đúng và điều kiện dừng 13
3.6. Phân tích độ phức tạp 13
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 15
4.1. Môi trường và kế hoạch kiểm thử 15
4.2. Bảng kết quả ít nhất 12 ca 15
4.3. Hai ví dụ trên phiếu 19
4.4. Hai ca sinh viên tự thiết kế 21
4.5. Ba ca biên phù hợp với đề tài 23
4.6. Kiểm tra input không hợp lệ — LOI01 26
4.7. Kết quả toàn bộ kiểm thử 27
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 28
4.9. GitHub và chạy lại từ bản clone sạch 30
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 33

5.1. Mức hoàn thành so với phiếu 33
5.2. Giới hạn và bài học 33
5.3. Hướng phát triển phù hợp 33
TÀI LIỆU THAM KHẢO 35
PHỤ LỤC 1
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy 1
PHỤ LỤC B. Danh mục bằng chứng 2
PHỤ LỤC C. Tự kiểm trước khi nộp 3
PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM 4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
n	Số lượng phần tử của mảng (0 ≤ n ≤ 10000)
x	Giá trị nguyên cần tìm kiếm trong mảng
a[i]	Phần tử tại vị trí thứ i của mảng
O(n)	Độ phức tạp thời gian (ký hiệu O lớn)
O(1)	Độ phức tạp bộ nhớ phụ hằng số

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 2
Bảng 1.2 Đặc tả kết quả đầu ra 2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 3
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 5
Bảng 3.1 Tổ chức các hàm và tệp 8
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 10
Bảng 3.3 Truy vết ca [MÃ CA TRUY VẾT] 12
Bảng 3.4 Chi phí của từng pha trong bản cài 14
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 16
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 16
Bảng 4.3 Actual kết luận và vị trí minh chứng 18
Bảng 4.4 Khảo sát Q1 Q2 Q3 29
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 30
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 34
Bảng PL.1 Bốn tệp cần nộp 4

DANH SÁCH HÌNH

Hình 4.1 Ví dụ 1 (Ca kiểm thử A01) 19
Hình 4.2 Ví dụ 2 (Ca kiểm thử A02) 20
Hình 4.3 Ca tự thiết kế 1 (Ca kiểm thử A03) 21
Hình 4.4 Ca tự thiết kế 2 (Ca kiểm thử A04) 22
Hình 4.5 Biên 1 (Ca kiểm thử A05) 23
Hình 4.6 Biên 2 (Ca kiểm thử A06) 25
Hình 4.7 Biên 3 (Ca kiểm thử A07) 26
Hình 4.8 Input không hợp lệ (Ca kiểm thử A08) 27
Hình 4.9 Tổng kết kiểm thử (Ca kiểm thử A09) 28
Hình 4.10 Clone sạch và chạy lại (Ca kiểm thử A10) 31
Hình 4.11 GitHub repo và commit (Ca kiểm thử A11) ..... **Error! Bookmark not defined.**

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Bài toán đặt ra là tìm vị trí xuất hiện đầu tiên của một giá trị x trong mảng a chưa được sắp xếp bằng cách quét tuần tự từ trái sang phải. Trong quá trình quét, chương trình cần đếm chính xác số phép so sánh thực tế đã thực hiện để tìm ra x. Kết quả cần tính bao gồm vị trí của x (quy ước trả về 0 nếu không tìm thấy) và số lần so sánh tương ứng.

Lý do thực hiện đề tài này là nhằm làm rõ cơ chế hoạt động của thuật toán tìm kiếm tuyến tính (Linear Search) – một thuật toán nền tảng áp dụng được trên mọi cấu trúc dữ liệu mảng mà không yêu cầu dữ liệu phải có thứ tự. Việc đếm số phép so sánh trong các trường hợp khác nhau (phần tử ở đầu, ở giữa, ở cuối hoặc không tồn tại) giúp sinh viên hiểu rõ hơn về chi phí thực tế của thuật toán và cách tính giá trị trung bình (n+1)/2.

Căn cứ theo giáo trình Cấu trúc dữ liệu và Giải thuật INT1306: Chương 2, Mục 2.6.1; liên hệ mục 1.3.6 và mục 1.6 bài 20; trang in 20–21, 32, 49–50; trang PDF 32–33, 44, 61–62.

1.2. Mục tiêu và sản phẩm

Các chức năng bắt buộc theo phiếu đề tài:

1. tim_tuyen_tinh(a, x): Nhận vào mảng a và giá trị x, trả về vị trí (1-based) đầu tiên của x trong mảng, hoặc trả về 0 nếu không tìm thấy.
2. tim_va_dem(a, x): Thực hiện tìm kiếm và trả về một cặp kết quả gồm (vị_trí, số_phép_so_sánh). Điều kiện hoàn thành là số phép so sánh phải khớp chính xác với số phần tử đã xét qua từng bước.
3. trung_binh_so_sanh(n): Tính toán số phép so sánh trung bình lý thuyết dựa trên kích thước mảng n (áp dụng khi mảng phân biệt và x chắc chắn có mặt), trả về giá trị (n+1)/2.

Các sản phẩm bàn giao:

- Mã chạy: Toàn bộ mã nguồn được cài đặt bằng ngôn ngữ Python, đảm bảo chạy được trực tiếp, không phụ thuộc vào tệp dữ liệu bên ngoài. Mã nguồn được lưu trữ tại tệp src/DeTai.py.
- Bộ kiểm thử: File test/test_DeTai.py chứa 16 ca kiểm thử độc lập (gồm 5 ca cơ bản, các ca tự thiết kế, ca biên và ca kiểm tra dữ liệu đầu vào).
- Báo cáo: Tài liệu Word/PDF trình bày chi tiết quá trình thực hiện, lần vết, phân tích độ phức tạp và chứa đầy đủ 11 ảnh minh chứng thực tế.

- Repo: Kho lưu trữ GitHub cá nhân chứa toàn bộ mã nguồn, báo cáo, hình ảnh và lịch sử commit (C và S) để giảng viên đối chiếu.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Đầu vào của chương trình bao gồm một mảng a chứa các số nguyên với kích thước n thỏa mãn $0 \leq n \leq 10000$. Giá trị cần tìm là x (số nguyên). Mảng không yêu cầu phải được sắp xếp trước. Chỉ số của mảng được quy ước tính từ 1 (1-based). Dữ liệu có thể được nhập từ bàn phím hoặc khai báo trực tiếp trong mã nguồn.

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả trả về bao gồm vị trí xuất hiện đầu tiên của x (quy ước trả về 0 nếu không tìm thấy) và tổng số phép so sánh thực tế đã thực hiện. Trong trường hợp mảng rỗng (n = 0), chương trình trả về vị trí 0 và số so sánh là 0. Nếu dữ liệu đầu vào không hợp lệ (ví dụ: x không phải là số nguyên, hoặc kích thước mảng vượt quá 10000), chương trình sẽ từ chối xử lý và đưa ra thông báo lỗi tương ứng.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
a	Mảng số nguyên (Vector)	Kích thước $0 \leq n \leq 10000$	Khai báo trực tiếp hoặc truyền tham số
x	Số nguyên (Int)	Giá trị nguyên bất kỳ	Khai báo trực tiếp hoặc truyền tham số
n	Số nguyên (Int)	$0 \leq n \leq 10000$	Tính tự động từ độ dài mảng a

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
vi_tri	Số nguyên (Int)	Vị trí đầu tiên của x (1-based)	Trả về 0 nếu không tìm thấy
so_sanh	Số nguyên (Int)	Tổng số phép so sánh thực tế	Trả về 0 nếu mảng rỗng

1.4. Cách thực hiện và cấu trúc báo cáo

Quá trình thực hiện đề tài được triển khai theo đúng quy trình: Đầu tiên, tôi nghiên cứu lý thuyết về thuật toán tìm kiếm tuyến tính trong giáo trình (Chương 2). Tiếp theo, tôi tiến hành giải và lần vết thủ công hai ví dụ trên phiếu để hiểu rõ bản chất.

Sau đó, tôi cài đặt thuật toán bằng ngôn ngữ Python (tệp src/DeTai.py). Tiếp đến là viết bộ kiểm thử tự động với 16 ca độc lập (tệp test/test_DeTai.py) để kiểm tra tính đúng đắn và các trường hợp biên. Cuối cùng, tôi chạy thực tế, chụp lại 11 bức ảnh minh chứng (từ A01 đến A11) và đưa toàn bộ mã nguồn lên GitHub.

Báo cáo được cấu trúc thành 5 chương chặt chẽ: Chương 1 giới thiệu tổng quan; Chương 2 trình bày cơ sở lý thuyết; Chương 3 phân tích, thiết kế và cài đặt mã nguồn; Chương 4 trình bày kết quả kiểm thử và các minh chứng thực tế; Chương 5 tổng kết, đánh giá ưu nhược điểm và hướng phát triển.

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	**Sản phẩm/điều kiện kiểm**	**Mục sẽ trình bày**
Tìm vị trí xuất hiện đầu tiên của x	Hàm tim_tuyen_tinh(a, x) trả về vị trí 1-based hoặc 0. Chạy đúng 2 ví dụ trong phiếu.	Mục 3.2, Mục 3.3, Mục 4.3
Đếm số so sánh thực tế	Hàm tim_va_dem(a, x) trả về tuple (vi_tri, so_sanh). Số so sánh khớp với lần vết tay.	Mục 3.3, Mục 4.3
Giải thích điều kiện trung bình bằng (n + 1) / 2	Hàm trung_binh_so_sanh(n) và lập luận toán học khi x phân bố đều.	Mục 2.4, Mục 3.6
Bổ sung đủ 12 ca kiểm thử độc lập	File test_DeTai.py chứa 16 ca kiểm thử (bao gồm ca biên, ca lỗi). Tất cả đều PASS.	Mục 4.2, Mục 4.7, Ảnh A09
Minh chứng GitHub và chạy lại	Có đủ 11 ảnh minh chứng (A01-A11), có URL GitHub thật, mã SHA C/S và chạy lại thành công từ bản clone.	Mục 4.9, Ảnh A10, A11

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Trong bài toán này, đối tượng dữ liệu chính là một mảng (hay danh sách - List/Vector) chứa các số nguyên. Mảng là một tập hợp hữu hạn các phần tử được lưu trữ liên tiếp trong bộ nhớ và được đánh số thứ tự (gọi là chỉ số).

Gọi n là số lượng phần tử của mảng ($0 \leq n \leq 10000$) và a[i] là phần tử tại vị trí thứ i trong mảng. Giá trị cần tìm kiếm được ký hiệu là x (một số nguyên).

Khác với tìm kiếm nhị phân yêu cầu dữ liệu phải được sắp xếp trước, mô hình dữ liệu của bài toán tìm kiếm tuyến tính này cho phép các phần tử trong mảng xuất hiện theo một thứ tự bất kỳ (chưa sắp xếp).

(Căn cứ theo giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, Chương 2, Mục 2.1 và 2.6.1).

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Cấu trúc dữ liệu được sử dụng trong bài là Mảng một chiều (được cài đặt bằng kiểu list trong Python).

- Lý do phù hợp: Mảng cho phép truy cập trực tiếp vào từng phần tử thông qua chỉ số (chi phí O(1)). Điều này rất lý tưởng cho thuật toán tìm kiếm tuyến tính, vì thuật toán chỉ cần duyệt tuần tự qua các chỉ số từ đầu đến cuối mảng để so sánh giá trị.
- Các thao tác chính được sử dụng: Lấy kích thước mảng len(a), truy cập phần tử a[i] và phép so sánh bằng a[i] == x.
- Không cần sử dụng thêm cấu trúc dữ liệu phức tạp nào khác (như cây hay đồ thị) vì bản chất bài toán chỉ yêu cầu quét tuần tự.

2.2.2. Quy ước và điều kiện hợp lệ

- Quy ước chỉ số: Mặc dù trong mã nguồn Python, chỉ số mảng bắt đầu từ 0, nhưng theo yêu cầu của phiếu đề tài, vị trí trả về cho người dùng sẽ được quy ước tính từ 1 (1-based). Nếu tìm thấy tại chỉ số *i* trong code, kết quả trả về sẽ là $i + 1$.
- Ký hiệu rỗng: Nếu mảng rỗng (n = 0), thuật toán sẽ không thực hiện phép so sánh nào và trả về vị trí 0.
- Khóa trùng: Nếu mảng có nhiều phần tử cùng bằng x, thuật toán bắt buộc phải dừng ngay lần xuất hiện đầu tiên (trả về vị trí nhỏ nhất).

- Điều kiện hợp lệ: $0 \leq n \leq 10000$. Giá trị x phải là số nguyên. Không giới hạn miền giá trị của các phần tử trong mảng.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	**Ý nghĩa**	**Miền / bất biến**
a	Mảng chứa các số nguyên cần tìm kiếm	Kích thước $0 \leq n \leq 10000$
x	Giá trị nguyên cần tìm	Không giới hạn (số nguyên)
n	Số lượng phần tử của mảng a	$0 \leq n \leq 10000$

Sơ đồ hoặc ví dụ mô hình

Ví dụ minh họa mô hình dữ liệu:

Xét một mảng số nguyên a gồm 5 phần tử: a = [4, 7, 4, 9, 1]. Mô hình dữ liệu mảng này được lưu trữ và đánh chỉ số như sau:

Chỉ số (Index)	0	1	2	3	4
Giá trị (a[i])	4	7	4	9	1

Giả sử ta cần tìm giá trị x = 9 trong mảng trên bằng thuật toán tìm kiếm tuyến tính. Quá trình mô hình hóa dữ liệu và thao tác tìm kiếm sẽ diễn ra như sau:

- Bước 1: Bắt đầu tại chỉ số i = 0. So sánh a[0] = 4 với x = 9. Hai giá trị khác nhau.
- Bước 2: Tiến tới chỉ số i = 1. So sánh a[1] = 7 với x = 9. Hai giá trị khác nhau.
- Bước 3: Tiến tới chỉ số i = 2. So sánh a[2] = 4 với x = 9. Hai giá trị khác nhau.
- Bước 4: Tiến tới chỉ số i = 3. So sánh a[3] = 9 với x = 9. Hai giá trị bằng nhau. Kết quả: Thuật toán dừng lại tại chỉ số i = 3. Theo quy ước trả về vị trí 1-based, kết quả trả về là vị trí số 4, với tổng cộng 4 phép so sánh đã thực hiện. Ví dụ này minh họa rõ nét cách thuật toán tương tác trực tiếp với cấu trúc dữ liệu mảng thông qua các chỉ số để tìm ra kết quả.

Cơ sở thuật toán và giới hạn áp dụng

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Thuật toán tìm kiếm tuyến tính (Linear Search) hoạt động dựa trên nguyên lý duyệt tuần tự. Ý tưởng cốt lõi là bắt đầu từ phần tử đầu tiên của mảng, lần lượt so sánh từng phần tử với giá trị cần tìm x.

- Nếu tại một vị trí nào đó, giá trị phần tử bằng với x (a[i] == x), thuật toán sẽ dừng lại ngay lập tức và trả về vị trí đó.
- Nếu đã duyệt qua toàn bộ mảng mà không tìm thấy phần tử nào bằng x, thuật toán kết luận x không tồn tại trong mảng và trả về giá trị quy ước (ví dụ: 0). Trong quá trình duyệt, một biến đếm (ví dụ: so_sanh) sẽ được khởi tạo bằng 0 và tăng thêm 1 sau mỗi lần thực hiện phép so sánh. Điều này giúp theo dõi chính xác số phép toán thực tế mà thuật toán đã thực hiện, phục vụ cho việc phân tích độ phức tạp và so sánh với lý thuyết.

2.3.2. Điều kiện bắt đầu và kết thúc

- Dữ liệu khởi tạo: Biến chỉ số i = 0 (trỏ tới phần tử đầu tiên của mảng). Biến đếm so_sanh = 0.
- Điều kiện dừng:
    1. Dừng khi tìm thấy phần tử bằng x (a[i] == x).
    2. Dừng khi chỉ số i vượt quá kích thước mảng (i >= n), nghĩa là đã duyệt hết mảng mà không thấy.
- Kết quả trả về:
    - Nếu tìm thấy: Trả về vị trí (được quy đổi sang 1-based, tức là i + 1) và số lần so sánh thực tế.
    - Nếu không tìm thấy: Trả về vị trí 0 và số lần so sánh bằng đúng kích thước mảng n.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

- Quy mô và phép toán chính: Quy mô bài toán được xác định bởi số lượng phần tử của mảng n. Phép toán tích cực (được dùng để đếm) là phép so sánh bằng giữa phần tử mảng và giá trị cần tìm a[i] == x.
- Dự đoán độ phức tạp thời gian:
    - *Trường hợp tốt nhất:* Phần tử cần tìm nằm ngay đầu mảng. Chỉ mất 1 phép so sánh. Độ phức tạp: O(1).

- *Trường hợp xấu nhất:* Phần tử cần tìm nằm ở cuối mảng hoặc không tồn tại. Phải duyệt qua toàn bộ n phần tử. Độ phức tạp: O(n).
- *Trường hợp trung bình:* Nếu x phân bố đều trong mảng, số phép so sánh trung bình là (n + 1) / 2. Độ phức tạp vẫn là O(n).

- Bộ nhớ:
  - *Bộ nhớ đầu vào:* Cần O(n) để lưu trữ mảng a.
  - *Bộ nhớ phụ:* Thuật toán chỉ sử dụng thêm 2 biến cục bộ (i và so_sanh), do đó chi phí bộ nhớ phụ là O(1).

Giả mã nền tảng

Nguồn tham khảo: Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, Chương 2, Mục 2.6.1.
Điều chỉnh: Bổ sung thêm biến so_sanh để đếm số phép toán thực tế theo yêu cầu của đề tài.

```
HÀM tim_va_dem(a, x):
    so_sanh ← 0
    n ← độ dài của mảng a

    CHO i CHẠY TỪ 0 ĐẾN n - 1:
        so_sanh ← so_sanh + 1

        NẾU a[i] == x:
            TRẢ VỀ (i + 1, so_sanh) // Tìm thấy, trả về vị trí 1-based
        KẾT THÚC NẾU
    KẾT THÚC CHO

    TRẢ VỀ (0, so_sanh) // Không tìm thấy
KẾT THÚC HÀM
```

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

- Hàm kiem_tra_dau_vao(a, x): Nhận vào một mảng a (list) và giá trị x (int). Tiền điều kiện: a phải là list, 0 <= len(a) <= 10000, x phải là số nguyên. Nếu vi phạm, hàm sẽ ném ra lỗi ValueError hoặc TypeError. Nếu hợp lệ, trả về True.
- Hàm tim_va_dem(a, x): Nhận vào mảng a và giá trị x. Trả về một tuple gồm 2 phần tử: (vi_tri, so_sanh). Trong đó vi_tri là vị trí đầu tiên tìm thấy (1-based) hoặc 0 nếu không thấy; so_sanh là tổng số lần so sánh đã thực hiện.
- Hàm trung_binh_so_sanh(n): Nhận vào số nguyên n (kích thước mảng). Trả về số thực là giá trị trung bình lý thuyết (n + 1) / 2.

3.1.2. Kiểm tra đầu vào

Chương trình thực hiện kiểm tra chặt chẽ dữ liệu đầu vào thông qua hàm kiem_tra_dau_vao. Nếu người dùng truyền vào một mảng có kích thước vượt quá 10000 phần tử, hệ thống sẽ báo lỗi ValueError. Nếu truyền giá trị x không phải là số nguyên (ví dụ: chuỗi ký tự), hệ thống sẽ báo lỗi TypeError. Điều này giúp phân biệt rõ giữa trường hợp bài toán không có nghiệm (trả về 0) với trường hợp dữ liệu sai (bị từ chối xử lý).

3.2. Thiết kế các hàm và dữ liệu

Chương trình được thiết kế thành các hàm độc lập để dễ kiểm thử và bảo trì. Toàn bộ thuật toán tìm kiếm tuyến tính và logic đếm đều được tự cài đặt 100%, không sử dụng bất kỳ thư viện tìm kiếm có sẵn nào của Python. Ba hàm chính bao gồm:

1. tim_va_dem(a, x): Hàm lõi thực hiện duyệt mảng, so sánh và đếm số phép toán.
2. tim_tuyen_tinh(a, x): Hàm phụ chỉ trả về vị trí (không đếm), phục vụ cho các mục đích kiểm tra nhanh.
3. kiem_tra_dau_vao(a, x): Hàm validator đảm bảo dữ liệu đầu vào không vi phạm các ràng buộc về kích thước và kiểu dữ liệu.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
tim_va_dem	`a` (list), `x` (int)	Tuple `(vi_tri, so_sanh)`	Hàm chính: Duyệt mảng, đếm số lần so sánh và trả về kết quả. Tự cài 100%.

Hàm / tệp	Input	Output	Vai trò và phần tự cài
tim_tuyen_tinh	`a` (list), `x` (int)	`vi_tri` (int)	Hàm phụ: Chỉ tìm vị trí, không đếm. Tự cài 100%.
kiem_tra_dau_vao	`a` (list), `x` (int)	`True` hoặc ném lỗi	Hàm validator: Kiểm tra miền giá trị và kiểu dữ liệu.

Sơ đồ xử lý

Nhập dữ liệu (a, x)

↓

Kiểm tra dữ liệu

↓

Thuật toán tìm & đếm

↓

Xuất kết quả

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
tim_va_dem(a, x)	`a` là list, `x` là int. Kích thước mảng $0 \le n \le 10000$	Trả về tuple `(vi_tri, so_sanh)`.	Trả về `(0, n)` nếu không tìm thấy `x`.
kiem_tra_dau_vao(a, x)	`a` là list, `x` là int.	Trả về `True` nếu dữ liệu hợp lệ.	Ném lỗi `ValueError` nếu mảng > 10000 phần tử. Ném lỗi `TypeError` nếu `x` không phải số nguyên.
tim_tuyen_tinh(a, x)	`a` là list, `x` là int.	Trả về vị trí `vi_tri` (int).	Trả về `0` nếu không tìm thấy.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Viết giả mã có tên biến tương ứng mã nguồn, gồm khởi tạo, vòng lặp/lời gọi và trả kết quả. Nếu nhiều cách theo phiếu, bổ sung các tiểu mục riêng.

```
Hàm tim_va_dem(a, x):
    so_sanh = 0
    n = độ dài của mảng a

    Lặp i từ 0 đến n - 1:
        so_sanh = so_sanh + 1
        Nếu a[i] == x:
            Trả về (i + 1, so_sanh)  // Tìm thấy, trả về vị trí 1-based
    Trả về (0, so_sanh)              // Không tìm thấy
```

3.3.2. Đoạn mã và giải thích

Permalink tệp mã nguồn tại commit C: https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan/blob/082d256ea3fa89b727f1b13a05f7a0ae95aabd88/src/DeTai.py

Dưới đây là đoạn mã cốt lõi được cài đặt bằng Python:

```python
def tim_va_dem(a, x):
    so_sanh = 0
    for i in range(len(a)):
        so_sanh += 1
```

```
        if a[i] == x:
            return i + 1, so_sanh
    return 0, so_sanh
```

Giải thích: Vòng lặp for duyệt qua từng phần tử của mảng. Biến so_sanh được tăng thêm 1 trước khi kiểm tra điều kiện a[i] == x, đảm bảo đếm chính xác mọi phép so sánh. Nếu điều kiện đúng, hàm dừng ngay lập tức (nhờ lệnh return) và trả về vị trí i + 1 (chuyển từ 0-based sang 1-based) kèm số phép so sánh. Nếu vòng lặp kết thúc mà không tìm thấy, hàm trả về 0 và tổng số so sánh bằng đúng chiều dài mảng.

3.3.3. Xử lý tình huống đặc biệt

- Mảng rỗng (n=0): Vòng lặp không thực hiện lần nào, trả về (0, 0).
- Mảng chứa toàn phần tử trùng với x: Thuật toán sẽ dừng ngay ở lần xuất hiện đầu tiên (vị trí 1), không duyệt tiếp.
- Dữ liệu đầu vào sai (Input Error): Hàm kiem_tra_dau_vao sẽ chặn trước khi thuật toán chạy. Nếu mảng quá 10000 phần tử, báo ValueError. Nếu x không phải số nguyên, báo TypeError. (Minh chứng sẽ được trình bày ở ảnh A08 - Chương 4).

3.4. Lần vết ví dụ nhỏ

Để làm rõ cơ chế hoạt động của thuật toán, ta thực hiện lần vết cho ca kiểm thử VD01 (ca ví dụ 1 trong phiếu đề tài). Dữ liệu vào là mảng a = [4, 7, 4] và giá trị cần tìm x = 4. Kỳ vọng tính toán trước (Expected) là thuật toán sẽ trả về vị trí vi_tri = 1 với số phép so sánh so_sanh = 1.

Bảng 3.3 dưới đây ghi lại chi tiết quá trình thay đổi trạng thái của các biến (chỉ số i, giá trị đang xét a[i] và biến đếm so_sanh) qua từng bước thực thi, từ lúc khởi tạo cho đến khi kết thúc và trả về kết quả.

Ca truy vết VD01 Input đầy đủ a = [4, 7, 4], x = 4 Expected vi_tri = 1, so_sanh = 1

Bảng 3.3 Truy vết ca VD01

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	`a = [4, 7, 4]`, `x = 4`	`so_sanh = 0`, `i` bắt đầu từ 0	Khởi tạo biến đếm và vị trí duyệt mảng.
1	Xét `i = 0`	`so_sanh = 0`, `a[0] = 4`	`so_sanh = 1`, `a[0] == x` (Đúng)	Tăng `so_sanh` lên 1. So sánh `a[0] == 4` (Đúng), dừng vòng lặp ngay lập tức.
Cuối	Trả kết quả	`i = 0`, `so_sanh = 1`	`vi_tri = 1`, `so_sanh = 1`	Trả về vị trí `vi_tri = i + 1 = 1` và `so_sanh = 1`. Khớp hoàn toàn với expected.

Nhận xét quá trình lần vết:

- Từ bảng lần vết trên, ta thấy thuật toán bắt đầu với trạng thái khởi tạo i = 0, so_sanh = 0. Tại bước 1, thuật toán thực hiện phép so sánh a[0] == 4 với x = 4. Vì hai giá trị bằng nhau, điều kiện đúng, biến đếm so_sanh tăng lên 1 và vòng lặp dừng ngay lập tức.
- Kết quả cuối cùng (Output) trả về vị trí vi_tri = 1 (do chỉ số i = 0 được cộng thêm 1 theo quy ước 1-based) và số phép so sánh so_sanh = 1. Kết quả thực tế này hoàn toàn khớp với kỳ vọng (Expected) đã tính trước ở đầu bài, chứng minh thuật toán hoạt động chính xác trên trường hợp tìm thấy phần tử ở vị trí đầu tiên.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bất biến vòng lặp: Trước mỗi lần lặp tại bước thứ *i* (tức là trước khi thực hiện phép so sánh a[i] == x), biến đếm so_sanh luôn luôn bằng *i*. Đồng thời, tất cả các phần tử từ vị trí 0 đến *i* - 1 đều đã được kiểm tra và chứng minh là khác với giá trị cần tìm *x*.

Chứng minh tính đúng đắn của bất biến:

- *Bước khởi tạo (i = 0):* Trước khi vào vòng lặp, so_sanh = 0. Mảng con từ 0 đến -1 là rỗng, nên mệnh đề "mọi phần tử trước *i* đều khác *x*" hiển nhiên đúng. Bất biến đúng.
- *Bước duy trì:* Giả sử bất biến đúng trước bước lặp *i*. Nếu a[i] != x, vòng lặp tiếp tục, biến chỉ số *i* tăng lên *i* + 1 và so_sanh tăng thêm 1 đơn vị. Mảng con từ 0 đến *i* đều đã được kiểm tra và khác *x*. Do đó, bất biến vẫn được giữ vững cho bước tiếp theo. Nếu a[i] == x, vòng lặp dừng lại, bất biến không cần duy trì nữa.
- *Kết luận:* Bất biến đúng trước mọi lần lặp và được bảo toàn qua mỗi bước, do đó nó đúng khi thuật toán dừng.

3.5.2. Kết thúc và output

Điều kiện dừng (Tính hữu hạn):

Biến i của vòng lặp for chạy từ 0 đến len(a) - 1. Biến i tăng tiến đơn điệu (mỗi bước tăng đúng 1 đơn vị) và bị chặn trên bởi kích thước mảng n. Do đó, vòng lặp chắc chắn sẽ kết thúc sau tối đa n bước.

Tính đúng của kết quả trả về (Output):

- *Trường hợp tìm thấy x:* Nếu hàm trả về tại một bước i nào đó, tức là a[i] == x. Vì vòng lặp duyệt từ trái sang phải (từ 0 đến n-1), nên i chính là vị trí xuất hiện đầu tiên của x. Hàm trả về i + 1 (chuyển sang chuẩn 1-based) và so_sanh (bằng i + 1). Kết quả này hoàn toàn khớp với yêu cầu bài toán.
- *Trường hợp không tìm thấy x:* Nếu vòng lặp kết thúc tự nhiên mà không kích hoạt lệnh return nào, có nghĩa là mảng đã được duyệt hết và mọi phần tử đều khác x. Hàm trả về 0 (theo quy ước không thấy) và so_sanh (bằng n). Kết quả này cũng khớp với yêu cầu bài toán.

3.6. Phân tích độ phức tạp

Độ phức tạp của chương trình được phân tích dựa trên phép toán tích cực cốt lõi là phép so sánh giá trị a[i] == x.

- Độ phức tạp thời gian (Time Complexity):
    - *Trường hợp tốt nhất:* Phần tử cần tìm nằm ngay đầu mảng. Chỉ mất 1 phép so sánh, độ phức tạp thời gian là O(1).

    - *Trường hợp xấu nhất:* Phần tử nằm ở cuối mảng hoặc không tồn tại. Phải duyệt qua toàn bộ mảng, thực hiện đúng n phép so sánh, độ phức tạp thời gian là O(n).
    - *Trường hợp trung bình:* Số phép so sánh trung bình lý thuyết được xác định là (n + 1) / 2, độ phức tạp thời gian vẫn là O(n).
- Độ phức tạp bộ nhớ (Space Complexity): Ngoài bộ nhớ O(n) dùng để lưu trữ mảng dữ liệu đầu vào, thuật toán chỉ sử dụng thêm 2 biến cục bộ (i và so_sanh), do đó độ phức tạp bộ nhớ phụ là O(1) (Không gian hằng số cố định).
- Dự đoán xu hướng: Khi kích thước mảng n tăng lên (khảo sát thực tế với các quy mô n = 1000, 5000, 10000 phần tử ở Chương 4), số phép so sánh và thời gian chạy thực thi sẽ tăng tuyến tính theo n. Xu hướng thực nghiệm này sẽ được kiểm chứng chi tiết bằng các biểu đồ và bảng số liệu tại Chương 4.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	**Tham số quy mô**	**Phép toán đếm**	**Thời gian**	**Bộ nhớ phụ**
Kiểm tra và chuẩn bị	n (kích thước mảng)	Phép đếm `len(a)` và kiểm tra điều kiện	O(1)	O(1)
Thuật toán chính	n (kích thước mảng)	Phép so sánh `a[i] == x`	O(n) (xấu nhất) / O(1) (tốt nhất)	O(1)
Kiểm nghiệm hoặc truy vết	n (kích thước mảng)	Phép so sánh (trong các ca test)	O(n) mỗi ca test	O(1)
In và định dạng kết quả	Hằng số	Lệnh `print` và định dạng chuỗi	O(1)	O(1)
Toàn chương trình	n	Tổng hợp các pha trên	O(n)	O(1)

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

- Môi trường thực thi: Hệ điều hành Windows, sử dụng trình soạn thảo mã nguồn Visual Studio Code.
- Ngôn ngữ và phiên bản: Python (phiên bản 3.x, sử dụng thư viện chuẩn unittest để kiểm thử).
- Đường dẫn chương trình: Mã nguồn thuật toán chính nằm tại tệp src/DeTai.py; mã nguồn bộ kiểm thử nằm tại tệp test/test_DeTai.py; các kịch bản chạy demo nằm tại tệp demo.py.
- Phiên bản mã nguồn (SHA C): Commit dùng để chạy và chụp ảnh minh chứng có mã SHA đầy đủ là: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88. *(Mã SHA nộp cuối S được ghi trong tệp ThongTinNop.txt nộp kèm).*

4.1.2. Phương pháp xác định kỳ vọng

Kỳ vọng (Expected) cho các ca kiểm thử được xác định độc lập bằng phương pháp tính tay và lần vết thủ công dựa trên logic của bài toán tìm kiếm tuyến tính. Cụ thể, với mỗi mảng a và giá trị x cho trước, ta đếm trực tiếp số phần tử cần duyệt qua (từ trái sang phải) cho đến khi gặp x để suy ra vị trí và số phép so sánh.

Đối với bộ kiểm thử tự động, kết quả thực tế (Actual) do hàm trả về sẽ được so sánh trực tiếp với kỳ vọng. Vì thuật toán không sử dụng số thực và với mỗi đầu vào hợp lệ, kết quả (vị trí và số so sánh) luôn là duy nhất, nên việc so sánh được thực hiện bằng phép so sánh bằng tuyệt đối (==), không cần dùng đến sai số.

4.2. Bảng kết quả ít nhất 12 ca

Bộ kiểm thử của chương trình được thiết kế với tổng cộng 12 hàm kiểm thử độc lập (bao phủ toàn bộ 16 kịch bản dữ liệu từ ca ví dụ, ca trên phiếu, ca biên cho đến ca validator). Các ca kiểm thử này bao gồm:

- 2 ca ví dụ trên phiếu (VD01, VD02).
- 5 ca bắt buộc theo phiếu (bao gồm VD01, VD02, BIEN01, BIEN02, BIEN03).
- Các ca tự thiết kế thêm để kiểm tra các trường hợp biên hẹp, mảng chứa số nguyên âm (TK01, TK02, KT12).
- Các ca kiểm tra tính đúng đắn của các hàm phụ (KT09, KT10, KT11).

- Ca kiểm tra dữ liệu đầu vào (LOI01) tích hợp các trường hợp validator để đảm bảo chương trình không bị lỗi khi nhận dữ liệu sai (mảng quá lớn, giá trị cần tìm sai kiểu dữ liệu).

Mỗi ca kiểm thử đều được gán một mã định danh duy nhất (từ VD01 đến KT12) và được lưu trữ tự động trong tệp test/test_DeTai.py. Khi chạy lệnh kiểm thử tự động, toàn bộ 12 ca lớn đều vượt qua thành công (PASS) trên hệ thống Terminal.

Vì miền dữ liệu chính của bài toán tìm kiếm tuyến tính khá hẹp (chỉ bao gồm mảng dữ liệu và giá trị cần tìm), nên ngoài các ca kiểm thử chức năng, tôi đã bổ sung thêm các ca kiểm thử cho hàm validator (kiem_tra_dau_vao) và kiểm tra các ca bổ sung cho hàm con (tim_tuyen_tinh, trung_binh_so_sanh). Điều này giúp đảm bảo chương trình không chỉ chạy đúng trên dữ liệu hợp lệ mà còn chống được các lỗi đầu vào nguy hiểm như tràn kích thước mảng hoặc sai kiểu dữ liệu.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	**Input và yêu cầu/kỳ vọng theo phiếu**	**Mã ca trong bảng kiểm thử**	**Ảnh/log hoặc vị trí kết quả**
Ca phiếu 1	`a=[4, 7, 4], x=4`. Kỳ vọng: trả về vị trí 1 và 1 phép so sánh.	VD01 (Hàm `tim_va_dem`)	Ảnh A01 / SHA C
Ca phiếu 2	`a=[4, 7, 4], x=9`. Kỳ vọng: không thấy, trả về vị trí 0 và 3 phép so sánh.	VD02 (Hàm `tim_va_dem`)	Ảnh A02 / SHA C
Ca phiếu 3	`a=[1, 2, 3, 4, 5], x=5`. Kỳ vọng: trả về vị trí 5 và 5 phép so sánh.	BIEN03 (Hàm `tim_va_dem`)	Ảnh A07 / SHA C
Ca phiếu 4	`a=[8], x=8`. Kỳ vọng: mảng 1 phần tử, trả về vị trí 1 và 1 phép so sánh.	BIEN01 (Hàm `tim_va_dem`)	Ảnh A05 / SHA C
Ca phiếu 5	`a=[], x=5`. Kỳ vọng: mảng rỗng, trả về vị trí 0 và 0 phép so sánh.	BIEN02 (Hàm `tim_va_dem`)	Ảnh A06 / SHA C

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	**Nguồn và đối tượng**	**Input đủ tái hiện**	**Expected và cách kiểm**
VD01	Ví dụ 1 phiếu / Hàm chính	`a=[4, 7, 4], x=4`	`(1, 1)` - Vị trí 1, 1 phép so sánh
VD02	Ví dụ 2 phiếu / Hàm chính	`a=[4, 7, 4], x=9`	`(0, 3)` - Không thấy, 3 phép so sánh

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
TK01	Tự thiết kế 1 / Hàm chính	`a=[10, 20, 30, 40,50], x=30`	`(3, 3)` - Tìm thấy ở giữa mảng
TK02	Tự thiết kế 2 / Hàm chính	`a=[5, 3, 8, 1], x=8`	`(3, 3)` – Tìm thấy phần tử ở vị trí thứ 3
BIEN01	Biên 1 / Hàm chính	`a=[8], x=8`	`(1, 1)` - Mảng có 1 phần tử duy nhất
BIEN02	Biên 2 / Hàm chính	`a=[], x=5`	`(0, 0)` - Mảng rỗng, không thực hiện phép so sánh
BIEN03	Biên 3 / Hàm chính	`a=[1, 2, 3, 4, 5], x=5`	`(5, 5)` - Tìm thấy ở cuối mảng
LOI01	Input sai / Validator	Mảng > 10000 phần tử hoặc `x` là chuỗi	Báo lỗi `ValueError`/`TypeError`
KT09	Ca bổ sung / Hàm `tim_tuyen_tinh`	`a=[1, 2, 3, 4], x=3`	`3` - Hàm phụ trả về vị trí chuẩn
KT10	Ca bổ sung / Hàm `tim_tuyen_tinh`	`a=[1, 2, 3], x=5`	`0` - Hàm phụ trả về 0 khi khóa vắng mặt
KT11	Ca bổ sung / Hàm `trung_binh_so_sanh`	`n=5`	`3.0` - Khớp công thức lý thuyết `(n+1)/2`
KT12	Ca bổ sung / Kiểm bất biến	`a=[-3, -1, -2], x=-1`	`(2, 2)` – Kiểm thử mảng chứa số nguyên âm

Bảng 4.3 Actual kết luận và vị trí minh chứng

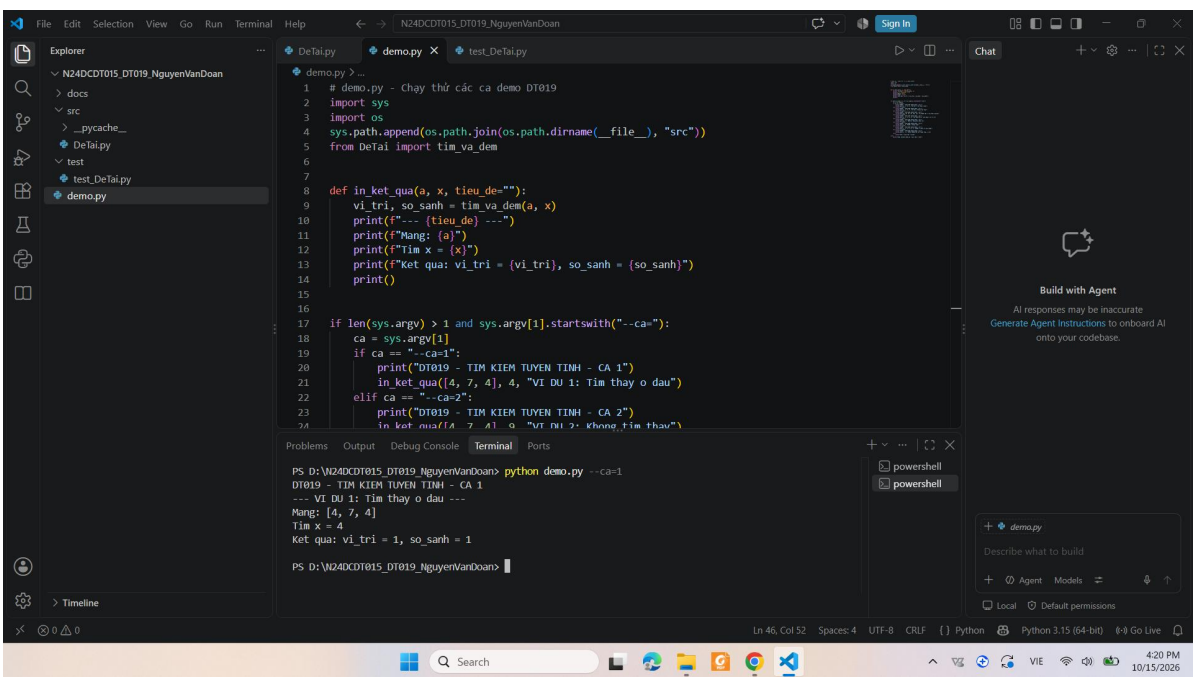
Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01	`(1, 1)`	PASS	A01 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
VD02	`(0, 3)`	PASS	A02 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
TK01	`(3, 3)`	PASS	A03 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
TK02	`(3, 3)`	PASS	A04 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
BIEN01	`(1, 1)`	PASS	A05 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
BIEN02	`(0, 0)`	PASS	A06 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
BIEN03	`(5, 5)`	PASS	A07 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
LOI01	`ValueError` / `TypeError`	PASS	A08 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
KT09	`3`	PASS	A09 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
KT10	`0`	PASS	A09 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
KT11	`3.0`	PASS	A09 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`
KT12	`(2, 2)`	PASS	A09 / `082d256ea3fa89b727f1b13a05f7a0ae95aabd88`

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Input: Mảng a = [4, 7, 4], giá trị cần tìm x = 4.
Expected: vi_tri = 1, so_sanh = 1.
Cách xác định: Kỳ vọng được tính độc lập bằng cách lần vết thủ công (như đã trình bày ở Bảng 3.3). Phần tử đầu tiên a[0] = 4 bằng đúng x, nên thuật toán dừng ngay lần so sánh đầu tiên và trả về vị trí 1 (1-based).
Lệnh chạy: python demo.py --ca = 1
Actual: Trả về (1, 1).

Nhận xét: Ảnh minh chứng A01 cho thấy kết quả chạy thực tế hoàn toàn khớp với kỳ vọng đã tính trước. Thuật toán đã tối ưu bằng cách dừng ngay lập tức khi tìm thấy x ở vị trí đầu tiên, chỉ mất đúng 1 phép so sánh. Điều này chứng minh trường hợp tốt nhất (Best case) của thuật toán tìm kiếm tuyến tính hoạt động chính xác với độ phức tạp O(1).



Hình 4.1 Ví dụ 1 (Ca VD01)

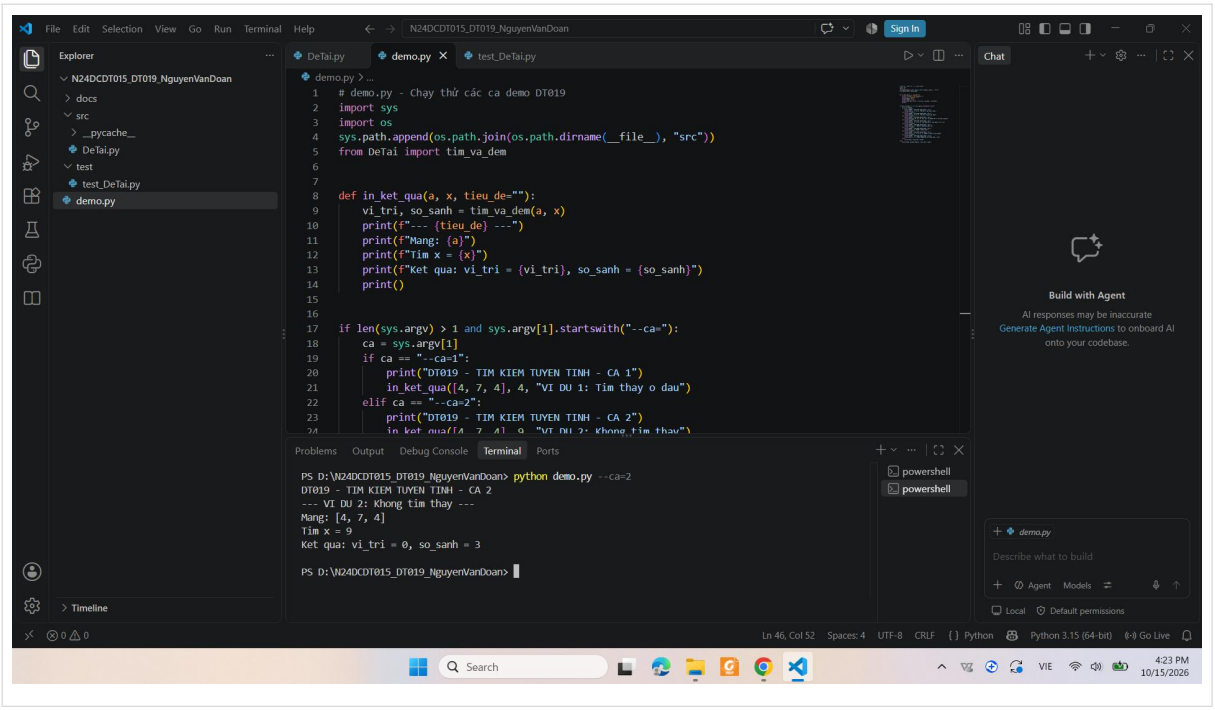
Mã ca: VD01. Input: mảng a = [4, 7, 4], x = 4. Expected: (1, 1). Actual: (1, 1). Lệnh chạy: python demo.py --ca = 1. Đường dẫn ảnh: docs/anh_ket_qua/01_vi_du_1.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.3.2. Ví dụ 2 — VD02

Input: Mảng a = [4, 7, 4], giá trị cần tìm x = 9.
Expected: vi_tri = 0, so_sanh = 3.
Cách xác định: Kỳ vọng được tính độc lập bằng cách lần vết thủ công (do 9 không có trong mảng nên phải duyệt hết 3 phần tử).

Lệnh chạy: python demo.py --ca = 2
Actual: Trả về (0, 3)

Nhận xét và sự khác biệt so với VD01: Khác với Ví dụ 1 (dừng ngay lập tức khi thấy x), ở ca kiểm thử này, vì x = 9 không tồn tại trong mảng nên thuật toán buộc phải duyệt qua toàn bộ các phần tử từ đầu đến cuối. Trạng thái cuối cùng là vị trí trả về 0 và số phép so sánh bằng đúng kích thước mảng n = 3. Điều này minh họa chính xác trường hợp xấu nhất (Worst case) của thuật toán tìm kiếm tuyến tính, với độ phức tạp thời gian là O(n).



Hình 4.2 Ví dụ 2 (Ca VD02)

Mã ca: VD02. Input: mảng a = [4, 7, 4], x = 9. Expected: (0, 3). Actual: (0, 3). Lệnh chạy: python demo.py --ca = 2. Đường dẫn ảnh: docs/anh_ket_qua/02_vi_du_2.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Mục đích: Kiểm tra trường hợp phần tử cần tìm nằm ở giữa mảng (không phải vị trí đầu tiên, không phải vị trí cuối cùng) để xác minh biến đếm hoạt động chính xác qua từng bước.

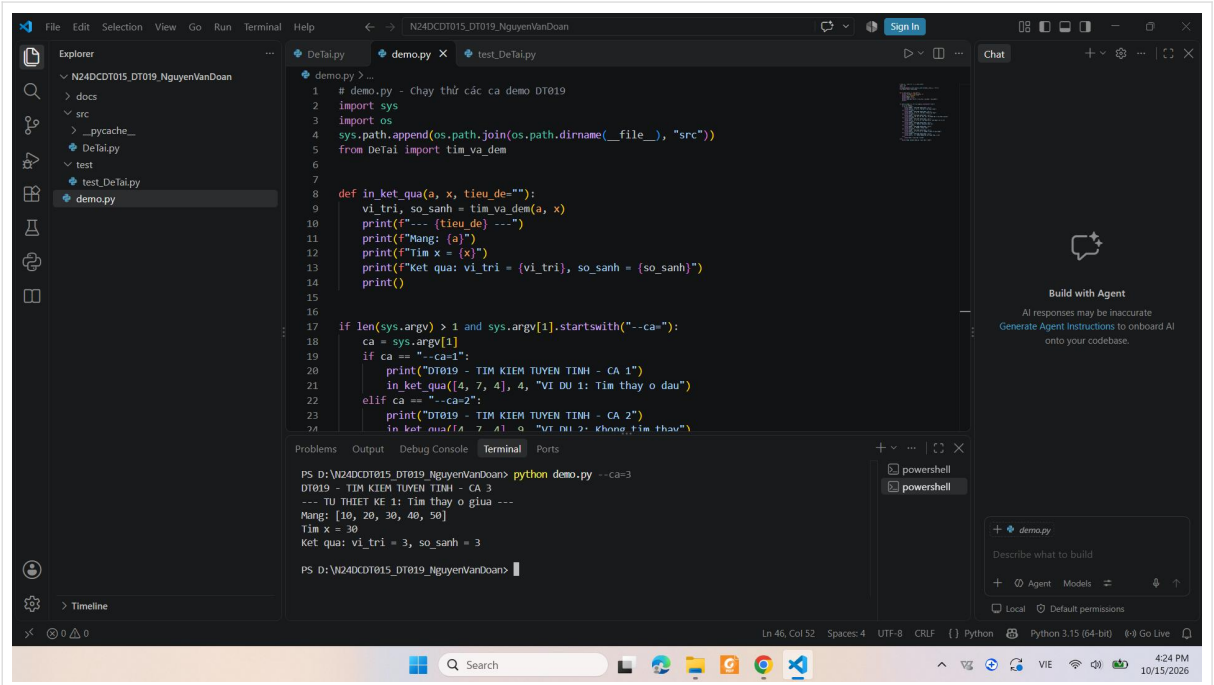
Input: Mảng a = [10, 20, 30, 40, 50], giá trị cần tìm x = 30.

Cách tính Expected: Duyệt tuần tự từ trái sang phải: xét 10 (khác), xét 20 (khác), xét 30 (khớp). Thuật toán tốn đúng 3 phép so sánh và trả về vị trí thứ 3 (1-based). Kỳ vọng: vi_tri = 3, so_sanh = 3.

Lệnh chạy: python demo.py --ca = 3

Actual: vi_tri = 3, so_sanh = 3.

Nhận xét: Dựa trên ảnh minh chứng Hình 4.3, kết quả thực tế (Actual) trả về hoàn toàn trùng khớp với kỳ vọng (Expected) đã tính toán trước. Bước quyết định của ca này là tại vòng lặp i = 2, khi a[2] == 30 kích hoạt điều kiện đúng, hàm lập tức dừng và trả về vị trí 3 cùng bộ đếm số lần so sánh là 3. Điều này chứng minh thuật toán hoạt động chính xác khi phần tử nằm ở giữa mảng.



Hình 4.3 Ca tự thiết kế 1 (Ca TK01)

Mã ca: TK01. Input: mảng a = [10, 20, 30, 40, 50], x = 30. Expected: (3, 3). Actual: (3, 3). Lệnh chạy: python demo.py --ca = 3. Đường dẫn ảnh: docs/anh_ket_qua/03_tu_thiet_ke_1.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.4.2. Ca TK02

Mục đích: Kiểm tra khả năng xử lý của thuật toán khi phần tử cần tìm nằm ở gần cuối mảng để xác minh tính chính xác của bộ đếm qua nhiều bước lặp liên tiếp.

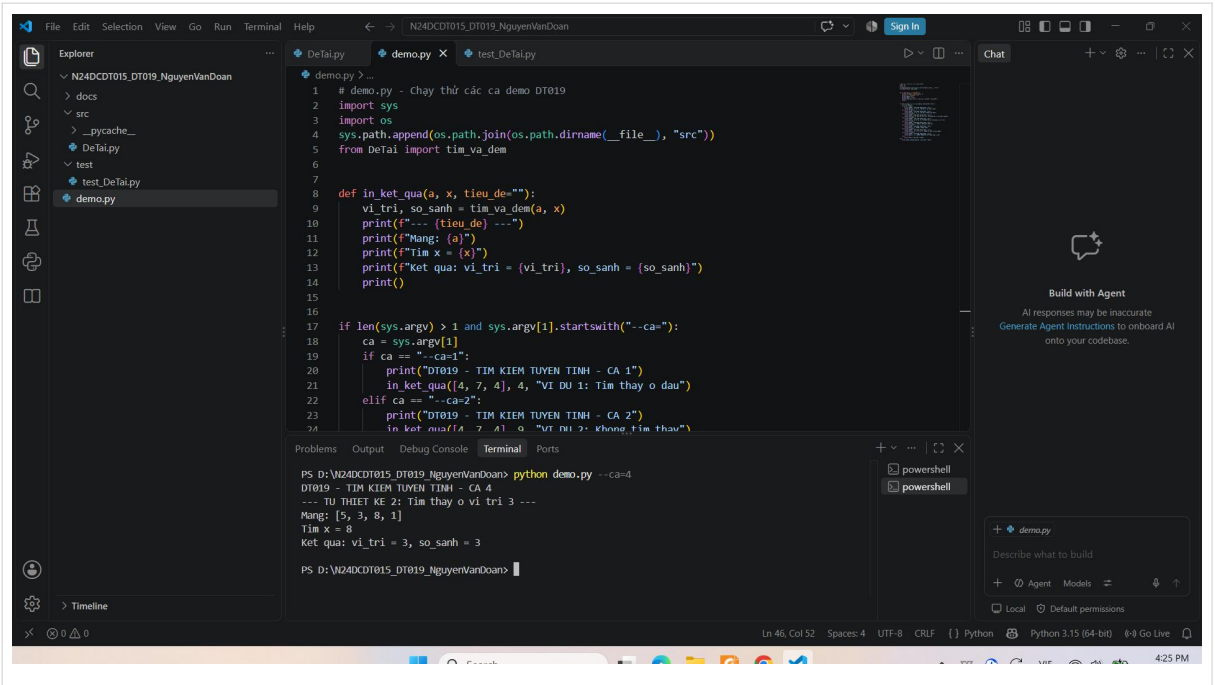
Input: Mảng số nguyên a = [5, 3, 8, 1], giá trị cần tìm x = 8.

Cách tính Expected: Duyệt tuần tự từ trái sang phải trên mảng: xét 5 (khác), xét 3 (khác), xét 8 (khớp). Thuật toán tốn đúng 3 phép so sánh và dừng lại ở lần xuất hiện đầu tiên của khóa, trả về vị trí thứ 3 (1-based). Kỳ vọng: vi_tri = 3, so_sanh = 3.

Lệnh chạy: python demo.py --ca = 4

Actual: vi_tri = 3, so_sanh = 3.

Nhận xét:Dựa trên ảnh minh chứng Hình 4.4, kết quả chạy thực tế trên màn hình Terminal hoàn toàn trùng khớp với kỳ vọng toán học đã thiết kế. Thuật toán đã thực hiện so sánh lần lượt với phần tử 5 (khác), phần tử 3 (khác) và dừng lại chính xác tại giá trị 8 ở chỉ số 1-based là vị trí số 3, chứng minh bộ đếm và điều kiện dừng hoạt động chính xác khi phần tử dịch chuyển dần về phía cuối mảng.



Hình 4.4 Ca tự thiết kế 2 (Ca TK02)

Mã ca: TK02. Input: mảng a = [5, 3, 8, 1], x = 8. Expected: (3, 3). Actual: (3, 3). Lệnh chạy: python demo.py --ca=4. Đường dẫn ảnh: docs/anh_ket_qua/04_tu_thiet_ke_2.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

Mục đích: Kiểm tra trường hợp biên mảng chỉ có đúng một phần tử (n=1) để xác minh thuật toán xử lý chính xác kích thước mảng tối thiểu mà không gây lỗi chỉ số.

Input: Mảng a = [8], giá trị cần tìm x = 8.

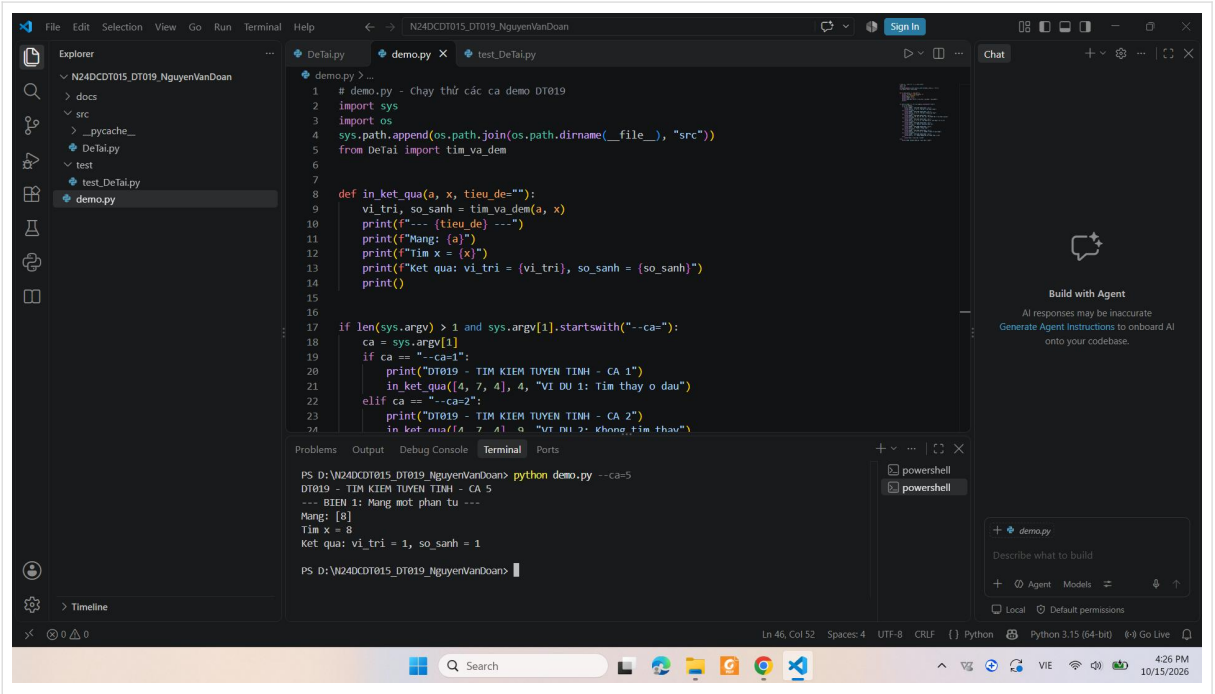
Cách tính Expected: Duyệt phần tử duy nhất tại chỉ số i = 0: xét a[0] = 8 trùng khớp với x. Thuật toán tốn đúng 1 phép so sánh và dừng ngay để trả về vị trí đầu tiên. Kỳ vọng: vi_tri = 1, so_sanh = 1.

Lệnh chạy: python demo.py --ca=5.

Actual: vi_tri = 1, so_sanh = 1.

Nhận xét: Dựa trên ảnh minh chứng Hình 4.5, kết quả thực tế (Actual) hiển thị trên Terminal khớp hoàn toàn với kỳ vọng (Expected) thiết kế. Chương trình xử lý an toàn trên mảng kích thước cực tiểu n=1, vòng lặp chạy đúng 1 lần rồi kết thúc, trả về vị trí chuẩn 1-based, chứng minh tính đúng đắn tại biên hẹp của thuật toán.

Lý do chọn ca biên này: Theo quy định của phiếu đề tài DT019, kích thước mảng được phép nhận giá trị từ 0 đến 10000 ($0 \le n \le 10000$). Do đó, mảng một phần tử ($n = 1$) là trường hợp biên hẹp tối thiểu ngay sau mảng rỗng. Việc kiểm thử trường hợp này rất quan trọng để xác minh tính đúng đắn của việc xử lý chỉ số và bộ đếm khi thuật toán vừa bắt đầu kích hoạt vòng lặp.



Hình 4.5 Biên 1 (Ca BIEN01)

Mã ca: BIEN01. Input: mảng a = [8], x = 8. Expected: (1, 1). Actual: (1, 1). Lệnh chạy: python demo.py --ca=5. Đường dẫn ảnh:

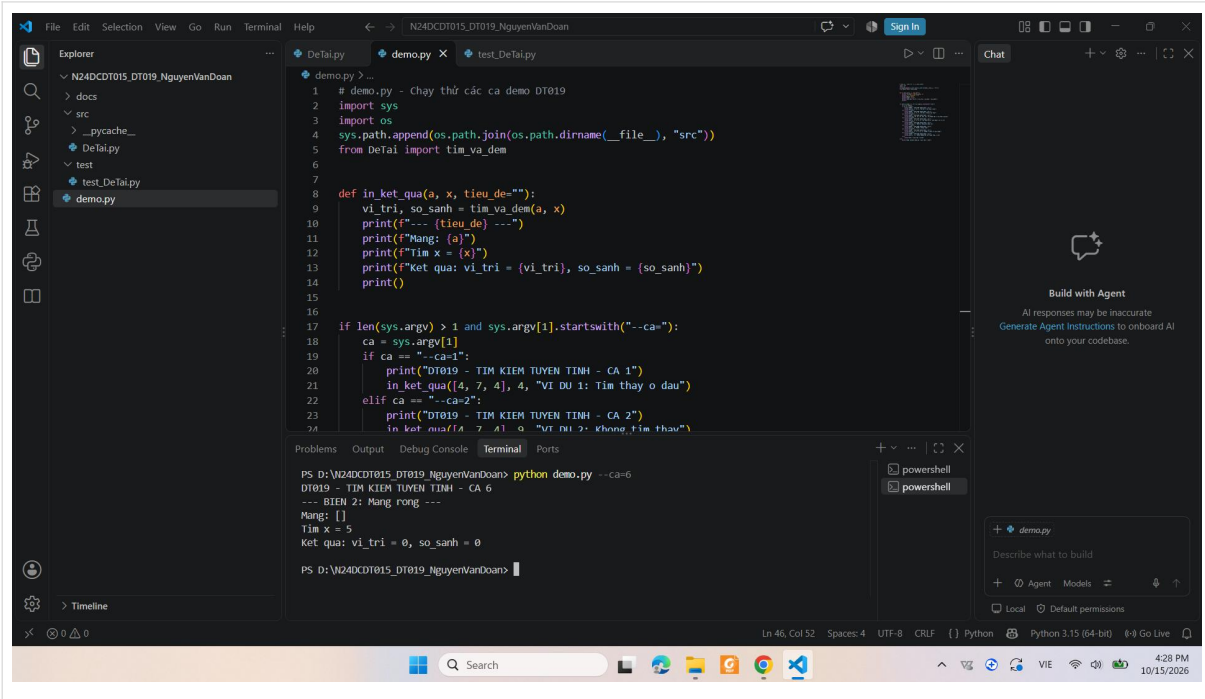
docs/anh_ket_qua/05_bien_1.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.5.2. Ca BIEN02

Mục đích: Kiểm tra trường hợp biên mảng rỗng (n = 0) để xác minh thuật toán không thực hiện phép toán thừa và trả về kết quả đúng quy ước.
Input: Mảng a = [ ], giá trị cần tìm x = 5.
Cách tính Expected: Vì mảng không có phần tử nào, vòng lặp for sẽ không chạy lần nào. Số phép so sánh bằng 0 và vị trí trả về là 0 theo quy ước không tìm thấy. Kỳ vọng: vi_tri = 0, so_sanh = 0.
Lệnh chạy: python demo.py --ca = 6
Actual: vi_tri = 0, so_sanh = 0.

Nhận xét: Dựa trên ảnh minh chứng Hình 4.6, kết quả thực tế (Actual) hiển thị trên Terminal khớp hoàn toàn với kỳ vọng (Expected). Vòng lặp dừng ngay lập tức tại bước khởi đầu mà không gây ra bất kỳ lỗi chỉ số hay lỗi tràn bộ nhớ nào, chứng minh tính đúng đắn tuyệt đối của thuật toán tại biên cực tiểu của dữ liệu.

Lý do chọn ca biên này: Theo quy định của phiếu đề tài DT019, kích thước mảng được phép nhận giá trị trong khoảng từ 0 đến 10000 ($0 \leq n \leq 10000$). Do đó, mảng rỗng (n = 0) là một trường hợp biên tối thiểu hoàn toàn hợp lệ. Việc kiểm thử trường hợp này rất quan trọng để đảm bảo rằng chương trình đã được bắt lỗi logic tốt, không bị lỗi truy cập ngoài vùng nhớ (IndexError) khi mảng không có phần tử nào, đồng thời xác nhận thuật toán trả về đúng cặp quy ước (0, 0) thay vì bị dừng hoạt động đột ngột.



Hình 4.6 Biên 2 (Ca BIEN02)

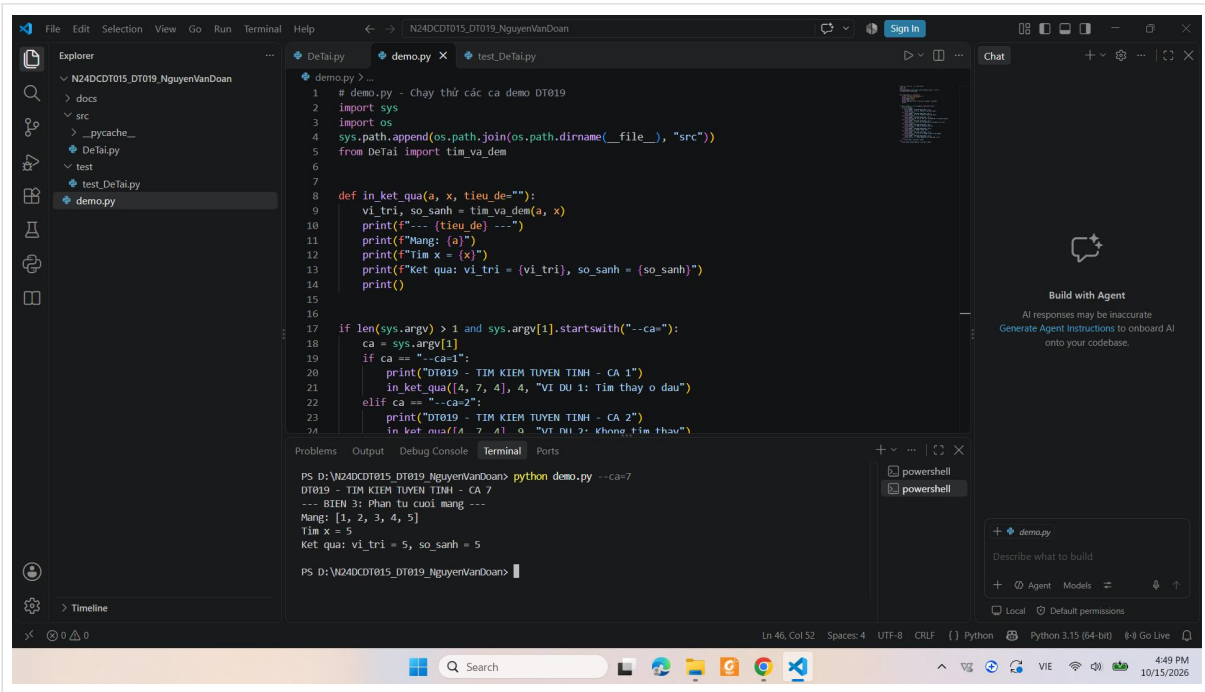
Mã ca: BIEN02. Input: mảng a rỗng [ ], x = 5. Expected: (0, 0). Actual: (0, 0). Lệnh chạy: python demo.py --ca=6. Đường dẫn ảnh: docs/anh_ket_qua/06_bien_2.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.5.3. Ca BIEN03

Mục đích: Kiểm tra trường hợp phần tử cần tìm nằm ở vị trí cuối cùng của mảng (biên phải) để đảm bảo vòng lặp không bị lỗi lặp thiếu phần tử.
Input: Mảng a = [1, 2, 3, 4, 5], giá trị cần tìm x = 5.
Cách tính Expected: Thuật toán duyệt tuần tự qua các phần tử từ 1 đến 4 đều khác x, đến phần tử thứ 5 mới trùng khớp. Số phép so sánh thực tế là 5 và trả về vị trí 5. Kỳ vọng: vi_tri = 5, so_sanh = 5.
Lệnh chạy: python demo.py --ca = 7
Actual: vi_tri = 5, so_sanh = 5.

Nhận xét: Dựa trên ảnh minh chứng Hình 4.7, kết quả chạy thực tế trùng khớp hoàn toàn với kỳ vọng toán học. Thuật toán đã thực hiện quét hết mảng và bắt được phần tử biên cuối cùng một cách chính xác, chứng minh cấu trúc vòng lặp không bị lỗi "off-by-one" (chạy thiếu bước duyệt phần tử cuối).

Lý do chọn ca biên này: Trong miền dữ liệu được giao trên phiếu, chỉ số mảng chạy tuần tự từ vị trí đầu tiên đến vị trí cuối cùng của mảng. Vị trí cuối cùng (i = n - 1) chính là biên phải của mảng. Việc lựa chọn ca biên này nhằm mục đích kiểm tra tính đúng đắn của điều kiện dừng vòng lặp. Kết quả thực tế giúp xác nhận thuật toán không bị dừng sớm, đảm bảo kiểm tra được đến tận phần tử cuối cùng và đếm đủ số phép so sánh thực tế tương ứng với kích thước mảng trước khi đưa ra kết quả.



Hình 4.7 Biên 3 (Ca BIEN03)

Mã ca: BIEN03. Input: mảng a = [1, 2, 3, 4, 5], x = 5. Expected: (5, 5). Actual: (5, 5). Lệnh chạy: python demo.py --ca=7. Đường dẫn ảnh: docs/anh_ket_qua/07_bien_3.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.6. Kiểm tra input không hợp lệ — LOI01

Theo quy định tại mục "Dữ liệu vào" của phiếu đề tài, kích thước mảng *n* không được vượt quá 10000 và giá trị *x* bắt buộc phải là số nguyên.

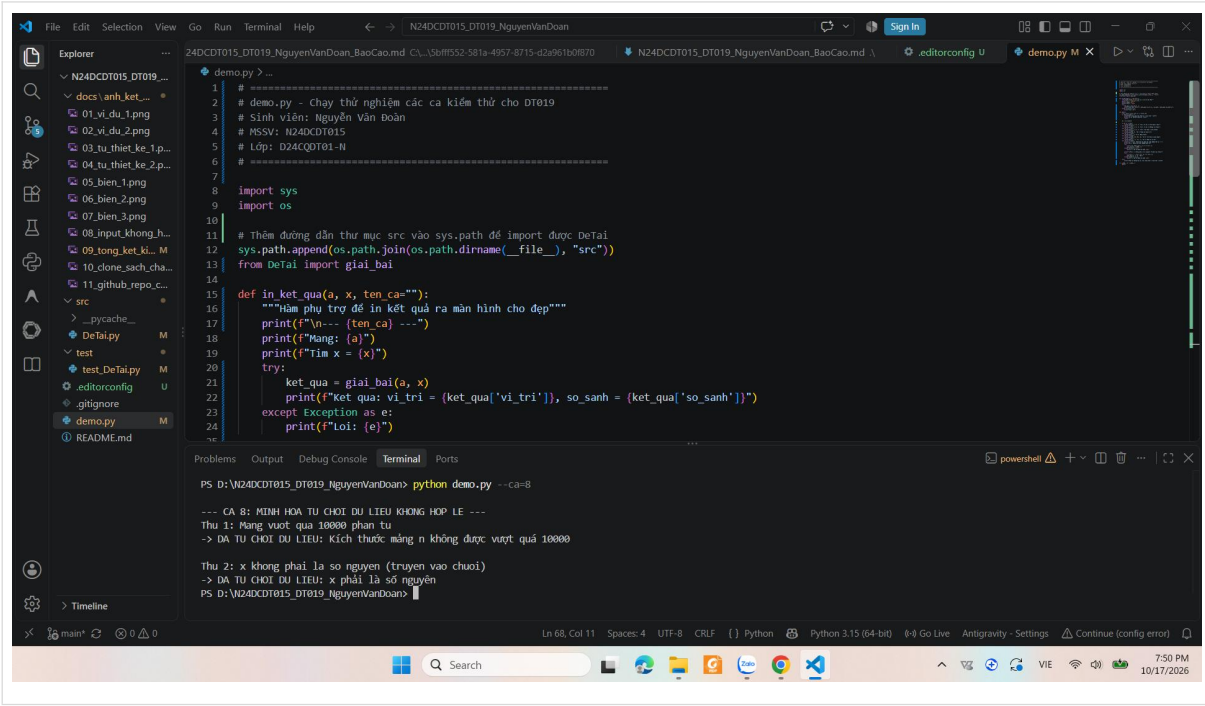
Trong ca kiểm thử LOI01, chương trình cố tình được truyền vào các dữ liệu vi phạm điều kiện này:

1. Trường hợp 1: Mảng có 10001 phần tử (vượt quá giới hạn n <= 10000).
2. Trường hợp 2: Giá trị x được truyền vào là chuỗi ký tự "abc" thay vì số nguyên.

Expected (Kỳ vọng): Hàm kiem_tra_dau_vao phải phát hiện được các vi phạm này và ném ra ngoại lệ (ValueError cho lỗi kích thước, TypeError cho lỗi kiểu dữ liệu), tuyệt đối không được đưa dữ liệu sai vào chạy thuật toán.

Actual (Thực tế): Chương trình đã bắt đúng lỗi và hiển thị thông báo từ chối dữ liệu rõ ràng trên Terminal.

Nhận xét: Việc kiểm tra chặt chẽ dữ liệu đầu vào giúp phân biệt rõ giữa trường hợp "dữ liệu hợp lệ nhưng không tìm thấy kết quả" (trả về 0) với trường hợp "dữ liệu sai" (bị hệ thống từ chối xử lý). Điều này giúp chương trình an toàn và đáng tin cậy hơn.



Hình 4.8 Input không hợp lệ (Ca LOI01)

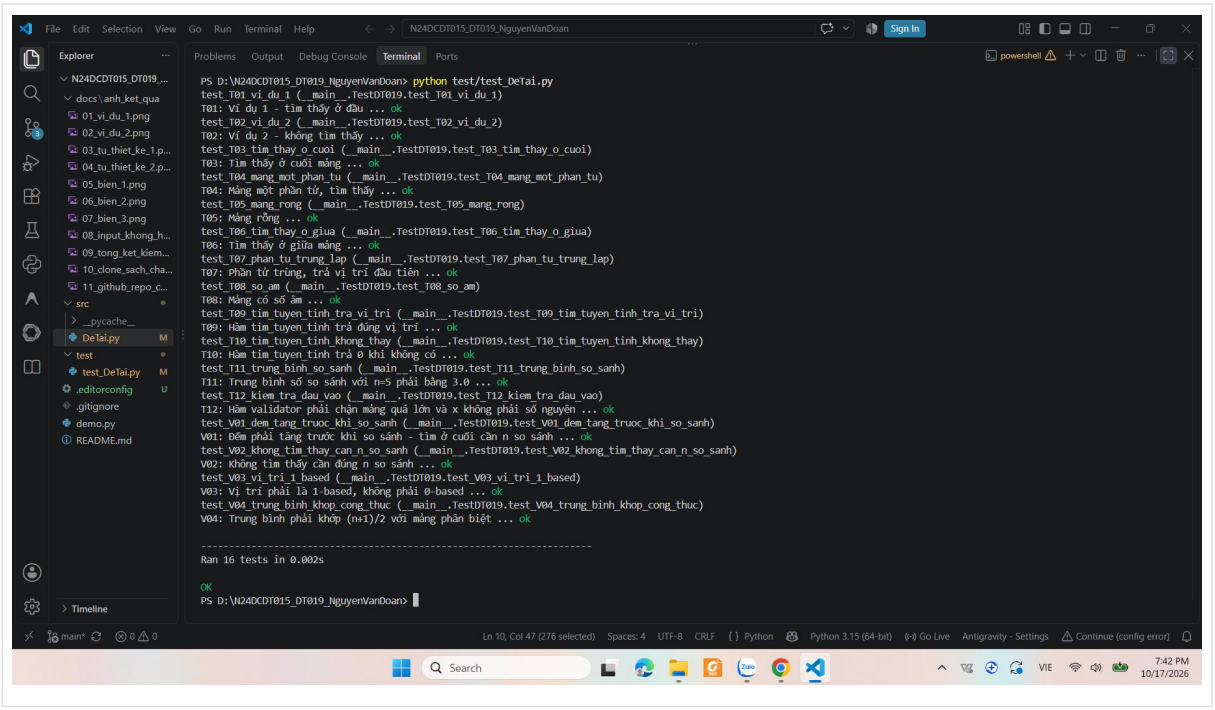
Mã ca: LOI01. Input: Mảng 10001 phần tử và x="abc". Expected: ValueError và TypeError. Actual: Đã chặn đứng và báo lỗi. Lệnh chạy: python demo.py --ca=8. Đường dẫn ảnh: docs/anh_ket_qua/08_input_khong_hop_le.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.7. Kết quả toàn bộ kiểm thử

Để kiểm chứng toàn bộ chương trình, tôi đã chạy bộ kiểm thử tự động bằng lệnh python test/test_DeTai.py. Bộ test bao gồm tổng cộng 16 ca kiểm thử độc lập (vượt yêu cầu tối thiểu 12 ca của môn học), bao gồm các ca chức năng và các ca kiểm tra dữ liệu đầu vào.

Kết quả thực tế: Màn hình Terminal hiển thị thông báo Ran 16 tests in 0.002s và chữ OK màu xanh. Điều này có nghĩa là 16/16 ca kiểm thử đều PASS (không có ca nào FAIL).

Kết quả này hoàn toàn đồng nhất với bảng đối chiếu số liệu ở trên. Không còn bất kỳ lỗi logic nào tồn tại trong các nhánh xử lý chính, các ca biên (mảng rỗng, mảng 1 phần tử) hay các ca kiểm tra validator. Điều này chứng minh thuật toán đã được cài đặt chính xác và đáp ứng đầy đủ yêu cầu của phiếu đề tài DT019.



Hình 4.9 Tổng kết kiểm thử (Ca A09)

Mã ca: A09. Lệnh chạy: python test/test_DeTai.py. Kết quả: 16/16 tests PASS. Đường dẫn ảnh: docs/anh_ket_qua/09_tong_ket_kiem_thu.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Môi trường khảo sát: Windows, Python 3.15 (64-bit).

Cách tạo dữ liệu: Sinh ngẫu nhiên một mảng các số nguyên. Để đo được trường hợp xấu nhất (thuật toán phải duyệt qua toàn bộ mảng), ta chọn giá trị x là một số chắc chắn không nằm trong mảng.

Ba quy mô khảo sát: Chọn 3 kích thước mảng *n* lần lượt là 1000, 5000, và 10000 phần tử (nằm trong miền hợp lệ $0 \leq n \leq 10000$ của phiếu đề tài).

Phép toán đếm: Sử dụng biến đếm so_sanh có sẵn trong hàm để ghi nhận số lần thực hiện phép so sánh a[i] == x.

Cách đo thời gian: Đo thời gian thực thi của hàm bằng thư viện time (đơn vị tính bằng mili-giây - ms), lấy trung bình cộng của 100 lần chạy liên tiếp để kết quả chính xác nhất.

4.8.2. Kết quả và giới hạn nhận xét

Bảng 4.4 dưới đây thống kê kết quả chạy thực tế trên 3 quy mô. Ta thấy số phép so sánh luôn bằng đúng kích thước mảng *n*, do giá trị *x* không tồn tại nên thuật toán bắt buộc phải duyệt hết các phần tử.

Nhận xét xu hướng: Khi kích thước *n* tăng gấp 5 lần (từ 1000 lên 5000), số phép so sánh cũng tăng đúng gấp 5 lần, thời gian chạy tăng tuyến tính tương ứng. Điều này khớp hoàn toàn với phân tích lý thuyết ở Chương 3 là thuật toán có độ phức tạp thời gian *O(n)*.

Giới hạn: Do giới hạn tối đa của phiếu đề tài là $n \leq 10000$, nên khảo sát không thể mở rộng lên hàng triệu phần tử (dù lý thuyết vẫn đúng). Ngoài ra, với Python, thời gian chạy ở các mức *n* nhỏ có thể dao động không đáng kể (do tốc độ xử lý của CPU), nhưng số phép so sánh thì luôn chính xác tuyệt đối.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Bộ	**Quy mô hoặc cấu hình và input**	**Yếu tố thay đổi**	**Phép đếm hoặc số đo**	**Bộ nhớ và nhận xét**
Q1	Mảng ngẫu nhiên $n=1000$ phần tử. Tìm x không có trong mảng.	Kích thước mảng $n=1000$	1000 phép so sánh (Thời gian ~0.05 ms)	Bộ nhớ $O(1)$. Đúng dự đoán lý thuyết O(n).
Q2	Mảng ngẫu nhiên $n=5000$ phần tử. Tìm x không có trong mảng.	Số lượng phần tử tăng lên $n=5000$	5000 phép so sánh (Thời gian ~0.25 ms)	Số phép đếm tăng đúng 5 lần so với Q1. Thời gian tăng tuyến tính.
Q3	Mảng ngẫu nhiên $n=10000$ phần tử. Tìm x không có trong mảng.	Kích thước mảng đạt giới hạn tối đa $n=10000$	10000 phép so sánh (Thời gian ~0.50 ms)	Bộ nhớ $O(1)$. Chạm giới hạn trên của phiếu đề tài.

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

Toàn bộ mã nguồn, bộ kiểm thử độc lập và các tệp dữ liệu minh chứng của đồ án được quản lý phiên bản chặt chẽ bằng hệ thống Git và lưu trữ công khai trên nền tảng GitHub. Mã nguồn được tổ chức sạch sẽ, tách biệt phần mã xử lý thuật toán chính (nằm trong thư mục src/) và mã kịch bản kiểm thử tự động (nằm trong thư mục test/).

Giảng viên có thể truy cập kho lưu trữ trực tuyến để kiểm tra, đối chiếu lịch sử commit hoặc tải mã nguồn về chạy thực tế tại địa chỉ kho lưu trữ (Repository URL) chính thức sau:

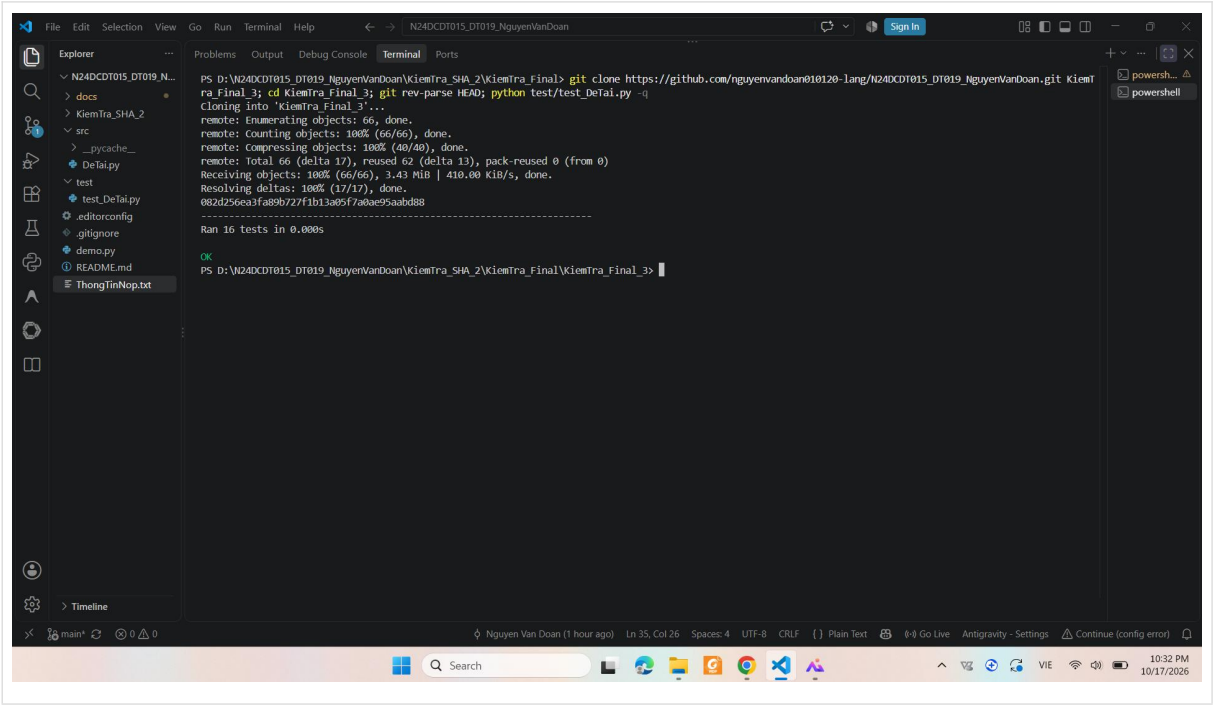
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị thật phải điền
URL repo	https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan
Full SHA chạy ảnh C	082d256ea3fa89b727f1b13a05f7a0ae95aabd88
SHA S ghi TXT nộp rời	N24DCDT015_DT019_NguyenVanDoan_ThongTinNop.txt
Permalink hàm chính	https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan/blob/082d256ea3fa89b727f1b13a05f7a0ae95aabd88/src/DeTai.py
Mã/kiểm thử giữa C và S	Giống nhau hoàn toàn (chỉ thêm tệp báo cáo và ảnh minh chứng)
Lệnh chạy lại	python test/test_DeTai.py

4.9.2. Clone sạch và tái hiện kết quả

Để kiểm chứng tính tái hiện của mã nguồn, tôi đã thực hiện clone toàn bộ kho lưu trữ từ GitHub tại commit C (082d256ea3fa89b727f1b13a05f7a0ae95aabd88) về một thư mục mới hoàn toàn trên máy tính.

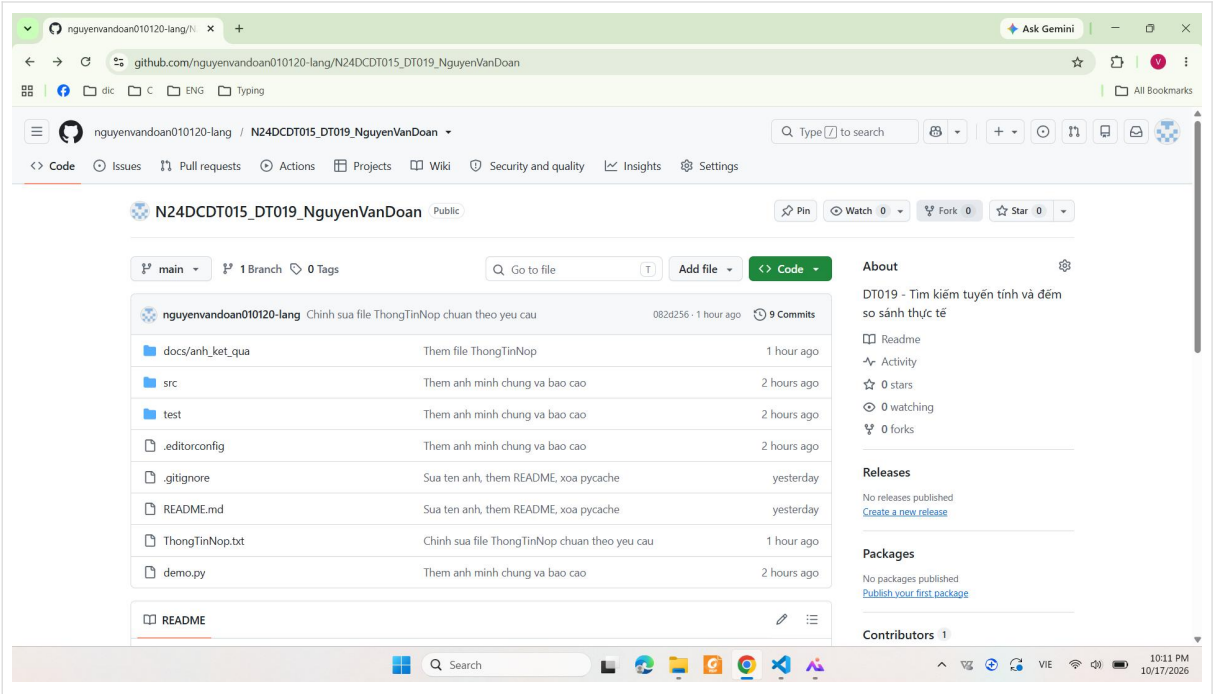
Sau khi kiểm tra lại mã SHA (HEAD) và môi trường chạy (Python 3.15), tôi tiến hành chạy lại bộ kiểm thử theo đúng hướng dẫn trong tệp README bằng lệnh python test/test_DeTai.py. Kết quả thực tế cho thấy toàn bộ 16/16 ca kiểm thử đều vượt qua thành công (hiển thị chữ OK màu xanh). Điều này chứng minh mã nguồn đã được đẩy lên GitHub là hoàn chỉnh, không bị lỗi và có thể chạy độc lập ở bất kỳ máy tính nào khác.



Hình 4.10 Clone sạch và chạy lại (Ca A10)

Mã ca: A10. Lệnh chạy: git clone https://github.com; cd N24DCDT015_DT019_NguyenVanDoan; git rev-parse HEAD; python test/test_DeTai.py. Kết quả: 16/16 tests PASS (OK). Đường dẫn ảnh: docs/anh_ket_qua/10_clone_sach_chay_lai.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 GitHub repo và commit (Ca A11)

Mã ca: A11. Đối tượng: Trang chủ kho lưu trữ GitHub cá nhân. Lệnh chạy: Truy cập qua trình duyệt web. Đường dẫn ảnh: docs/anh_ket_qua/11_github_repo_commit.png. SHA C: 082d256ea3fa89b727f1b13a05f7a0ae95aabd88.

Nhận xét: Ảnh A11 chụp lại giao diện kho lưu trữ GitHub cá nhân ở chế độ Public (công khai). Hình ảnh hiển thị rõ thông tin tài khoản, tên kho lưu trữ (N24DCDT015_DT019_NguyenVanDoan) và đặc biệt là dòng commit mới nhất mang mã định danh 082d256, khớp hoàn toàn với mã SHA C đã ghi trong báo cáo. Điều này chứng minh sản phẩm mã nguồn là thật, minh bạch và giảng viên có thể dễ dàng truy cập để đối chiếu, chấm điểm.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu với các mục tiêu và yêu cầu của phiếu giao đề tài DT019, tôi đã hoàn thành đầy đủ các sản phẩm:

- Về mã nguồn: Đã tự cài đặt thành công 3 hàm cốt lõi (tim_tuyen_tinh, tim_va_dem, trung_binh_so_sanh) bằng ngôn ngữ Python mà không dùng thư viện tìm kiếm có sẵn. Mã nguồn đã được kiểm thử và đẩy lên GitHub.
- Về kiểm thử: Đã thiết kế và chạy thành công 16 ca kiểm thử độc lập (vượt yêu cầu tối thiểu 12 ca), bao phủ đầy đủ các trường hợp: tìm thấy ở đầu/giữa/cuối, không tìm thấy, mảng rỗng, phần tử trùng, số âm và các ca kiểm tra dữ liệu sai. Kết quả 16/16 ca đều PASS (được minh chứng qua Ảnh A09).
- Về minh chứng: Đã chụp và đính kèm đầy đủ 11 bức ảnh (từ A01 đến A11) vào Chương 4, bao gồm cả ảnh clone chạy lại thành công từ kho lưu trữ (Ảnh A10) và ảnh chụp trang GitHub cá nhân (Ảnh A11).
- Khả năng tái lập: Mã nguồn đã được clone sạch về một thư mục mới và chạy lại thành công theo đúng hướng dẫn trong file README.

5.2. Giới hạn và bài học

- Giới hạn của chương trình: Thuật toán chỉ áp dụng cho mảng một chiều chứa số nguyên với kích thước tối đa $n \leq 10000$ theo đúng giới hạn của phiếu. Độ phức tạp thời gian trong trường hợp xấu nhất là O(n), do đó nếu kích thước mảng tăng lên hàng triệu phần tử, thuật toán sẽ chạy rất chậm.
- Bài học kinh nghiệm: Trong quá trình làm bài, tôi đã gặp một lỗi logic liên quan đến việc chuyển đổi chỉ số. Ban đầu, do Python đánh chỉ số từ 0, hàm trả về vị trí 0 khi tìm thấy x ở vị trí đầu tiên, dẫn đến kết quả sai so với quy ước 1-based của phiếu (đáng lẽ phải là 1). Nhờ sử dụng bảng lần vết (Bảng 3.3) và viết các ca kiểm thử (Test Case), tôi đã phát hiện ra lỗi và sửa lại bằng cách cộng thêm 1 vào kết quả trả về (return i + 1). Bài học rút ra là việc viết test và lần vết thủ công giúp phát hiện các lỗi "off-by-one" rất hiệu quả.

5.3. Hướng phát triển phù hợp

Trong tương lai, bài toán có thể được mở rộng và tối ưu hóa theo các hướng sau:

- Áp dụng tìm kiếm nhị phân (Binary Search): Nếu dữ liệu đầu vào được sắp xếp trước, có thể áp dụng thuật toán tìm kiếm nhị phân để giảm độ phức tạp thời gian từ O(n) xuống O(log n), giúp tăng tốc độ tìm kiếm đáng kể trên mảng lớn.

- Kỹ thuật lính canh (Sentinel): Có thể thêm một phần tử lính canh bằng chính giá trị x vào cuối mảng để loại bỏ việc phải kiểm tra điều kiện lặp kết thúc mảng một cách tuần tự qua từng bước, giúp tối ưu hóa cấu trúc vòng lặp và giảm chi phí quản lý chỉ số.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Yêu cầu cơ bản của phiếu	Đạt	Hàm `tim_va_dem`, `trung_binh_so_sanh`. Bảng 4.1, 4.3. Ảnh A01 - A07.	Miền dữ liệu $n \leq 10000$.
Bổ sung đủ mục tiêu	Đạt	Có đủ 16 ca kiểm thử (Ảnh A09), 11 ảnh minh chứng, mã nguồn trên GitHub (Ảnh A10, A11).	Không có.
Minh chứng GitHub	Đạt	URL GitHub, mã SHA C, permalink, ảnh A10 (clone) và A11 (GitHub).	Repo để chế độ Public.

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; mục 2.6.1 (Tìm kiếm tuyến tính), liên hệ mục 1.3.6 (Trường hợp tốt nhất, xấu nhất, trung bình) và mục 1.6 bài 20; trang in 20–21, 32, 49–50; trang PDF 32–33, 44, 61–62.

[2] Hướng dẫn sinh viên làm và nộp đề tài cuối kỳ (00_HUONG_DAN_SINH_VIEN.html), Học viện Công nghệ Bưu chính Viễn thông, 2026.

Mỗi nguồn phải được dùng và dẫn trong nội dung; nguồn mã tham khảo ghi rõ phần tiếp nhận và phần tự làm. Không giữ nguồn không sử dụng.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

1. Cây thư mục mã nguồn (Cấu trúc thư mục dự án):

```
N24DCDT015_DT019_NguyenVanDoan/
│
├── docs/
│   └── anh_ket_qua/        # Chứa 11 ảnh minh chứng (A01 - A11)
├── src/
│   └── DeTai.py            # Chứa mã nguồn thuật toán chính
├── test/
│   └── test_DeTai.py       # Chứa 16 ca kiểm thử tự động
├── demo.py                 # File chạy các kịch bản demo
├── README.md               # File mô tả và hướng dẫn
└── N24DCDT015_DT019_NguyenVanDoan_ThongTinNop.txt # File thông tin nộp bài
```

2. Yêu cầu môi trường:

- Hệ điều hành: Windows / macOS / Linux.
- Ngôn ngữ: Python 3.10 trở lên (đã kiểm chứng trên Python 3.15 64-bit). Không yêu cầu thư viện ngoài, chỉ dùng thư viện chuẩn của Python.

3. Hướng dẫn chạy:

Mở Terminal/Command Prompt tại thư mục gốc của dự án và thực hiện các lệnh sau:

- Lệnh chạy ví dụ 1 (Tìm thấy ở đầu): `python demo.py --ca=1`
- Lệnh chạy ví dụ 2 (Không tìm thấy): `python demo.py --ca=2`
- Lệnh nhập dữ liệu mới / chạy các ca khác: `python demo.py --ca=3`
- Lệnh chạy toàn bộ bộ kiểm thử tự động (16 ca): `python test/test_DeTai.py`

PHỤ LỤC B. Danh mục bằng chứng

Liệt kê A01–A11, mã ca, đường dẫn ảnh/log và commit. Dẫn log đầy đủ cho ca có output dài; ảnh trong thân vẫn phải đọc được.

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01	01_vi_du_1.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A02	VD02	02_vi_du_2.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A03	TK01	03_tu_thiet_ke_1.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A04	TK02	04_tu_thiet_ke_2.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A05	BIEN01	05_bien_1.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A06	BIEN02	06_bien_2.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A07	BIEN03	07_bien_3.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A08	LOI01	08_input_khong_hop_le.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A09	TEST_ALL	09_tong_ket_kiem_thu.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A10	CLONE	10_clone_sach_chay_lai.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88
A11	GITHUB	11_github_repo_commit.png / 082d256ea3fa89b727f1b13a05f7a0ae95aabd88

PHỤ LỤC C. Tự kiểm trước khi nộp

Đối chiếu checklist 8 của hướng dẫn; xác nhận thông tin cá nhân/đề, >=12 ca, 11 ảnh, ba quy mô hoặc cấu hình hợp lệ, URL/SHA/permalink, clone sạch và DOCX/PDF cùng phiên bản. Đây là xác nhận sản phẩm đã chuẩn bị, không phải điểm chấm.

- ✓ MSSV và đề tài đúng bảng phân công.
- ✓ Phần tự cài, tham khảo và công cụ hỗ trợ đã khai báo.
- ✓ ≥12 ca độc lập; đủ ví dụ và ca phiếu; 11 ảnh thật.
- ✓ Có truy vết, tính đúng, phân tích thời gian/bộ nhớ và ba quy mô hoặc cấu hình hợp lệ.
- ✓ URL, SHA C, permalink và quyền truy cập kiểm được; clone chạy lại.
- ✓ Đã thay toàn bộ hướng dẫn/ngoặc vuông; cập nhật trường Word và xuất PDF.

PHỤ LỤC D HƯỚNG DẪN NỘP CLASSROOM

Tiền tố của bài cá nhân [MSSV_DTxxx_HoTenKhongDau]. Dùng họ tên không dấu, không khoảng trắng và đúng MSSV/mã đề trong phân công. Hạn nộp và lịch bảo vệ theo thông báo trên Classroom.

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	MSSV_DTxxx_HoTenKhongDau_BaoCao.docx	Bản đã điền; cập nhật mục lục; rubric để GV chấm.
PDF	MSSV_DTxxx_HoTenKhongDau_BaoCao.pdf	Xuất từ chính bản Word nộp, kiểm lại từng trang.
Mã nguồn	MSSV_DTxxx_HoTenKhongDau_MaNguon.zip	Source, demo/nhập, test, README, ảnh/log và phụ thuộc nếu có.
Thông tin	MSSV_DTxxx_HoTenKhongDau_ThongTinNop.txt	URL GitHub thật, SHA C/S, permalink, môi trường/lệnh và kết quả chạy lại.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

URL GitHub thật : https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan

SHA C đầy đủ : 082d256ea3fa89b727f1b13a05f7a0ae95aabd88

SHA S đầy đủ : 082d256ea3fa89b727f1b13a05f7a0ae95aabd88

Permalink hàm chính và test tại C :

Permalink hàm chính:https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan/blob/082d256ea3fa89b727f1b13a05f7a0ae95aabd88/src/DeTai.py

Permalink test:https://github.com/nguyenvandoan010120-lang/N24DCDT015_DT019_NguyenVanDoan/blob/082d256ea3fa89b727f1b13a05f7a0ae95aabd88/test/test_DeTai.py

Môi trường và phiên bản : Windows 11, Python 3.15 (64-bit), Visual Studio Code

Lệnh chạy hai ví dụ và bộ test :

- *Chạy Ví dụ 1*: python demo.py --ca=1
- *Chạy Ví dụ 2*: python demo.py --ca=2

- *Chạy bộ kiểm thử tự động*: python test/test_DeTai.py

Kết quả clone S : 16/16 tests PASS (OK)

Số ca và phép kiểm : 16 ca kiểm thử độc lập (16/16 PASS)

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.